

变物性与收缩条件下药材烘干的传热传质模型

摘 要

药材烘干过程中，内部升温与水分迁移的快慢不同，物性变化和几何收缩又共同影响干燥终点。为求得圆柱形药材的径向温度、干基含水率及烘干时长，本文建立一维径向传热传质模型，依次引入变物性和实测收缩，以环形有限体积法离散空间，结合隐式时间积分与 Richardson 外推求解。

针对问题一，在固定半径下分别求解常热物性导热方程与含水率相关扩散方程。1800 s 时，中心和表面温度分别为 33.5753 °C、36.7856 °C，干基含水率分别为 2.5500 kg/kg、1.5102 kg/kg。预热阶段中心水分变化较小，失水主要发生在表层。

针对问题二，将题设物性写为局部温度与含水率的函数，联立求解温湿耦合方程。3 h 时，中心和表面温度分别为 49.8495 °C、49.9664 °C，含水率分别为 1.7662 kg/kg、1.0081 kg/kg。此时温度已接近均匀，水分仍存在明显径向梯度，表明升温完成后仍需持续干燥。

针对问题三，以附件 1 最后一小时的环境均值延拓长期边界，并以全域最大含水率低于 0.15 kg/kg 判定干燥完成。固定半径模型的临界时长为 57.4740 h，首个严格达标整分钟为 57.4833 h。不同长期环境设定下，临界时长约为 57.24—57.83 h，说明终点预测对环境延拓假设较为敏感。

针对问题四，由均匀收缩下的干物质与水分守恒建立移动边界模型，通过归一化径向坐标将变动区域映射到固定区间。计算得临界时长为 51.0920 h，首个严格达标整分钟为 51.1000 h。保持本问物性而固定初始半径时，对照时长为 129.8481 h，表明收缩对该模型的干燥进程具有较大影响。

网格加密、通量平衡和独立离散检验支持了数值结果的一致性，两类终点的相邻外推估计差均约为 1 s。所给时长仍受一维近似、有效传质边界及经验密度与收缩关系相容性的限制，属于上述假设下的模型预测。

关键词：药材烘干；传热传质；有限体积法；移动边界；干燥终点

1 问题重述与分析

1.1 任务及数据

题目研究长 25 cm、初始半径 2 cm 的圆柱形药材，其初始温度为 28 °C，初始干基含水率为 2.55 kg/kg。附件 1 提供烘房初期温度及含湿量，附件 2 提供药材半径随时间的变化。所求结果不仅包含平均干燥趋势，还包括指定物理半径处的局部温度和含水率，因此必须建立具有空间分辨能力的模型 [1]。

四问依次考察预热、变物性干燥、终点判定和几何收缩。第一问计算固定半径下的 30 min 预热过程；第二问从初始时刻采用附录 3 的物性，求解全程温湿分布并列出前三小时结果；第三问沿用第二问模型，确定各处含水率均低于 0.15 kg/kg 的时间；第四问引入附录 4 物性和实测半径，重新计算干燥终点。第二问独立从题设初值起算，第三问则是其长期延伸。

1.2 模型选择

题目要求同时给出局部分布和全域达标时间，仅用平均水分比的经验曲线难以完成这两项任务。因此，本文采用 Fourier 导热和 Fick 有效扩散描述内部输运，以表面对流边界连接烘房环境。圆柱物料的热质耦合模型 [2] 和非等温移动边界模型 [3] 为这一选择提供了参考，具体物性与观测输入均取自本题。

药材长径比为 6.25，本文将径向结果解释为圆柱中截面的分布，采用一维轴对称近似。由常物性到变物性，再到移动边界，逐层增加题目提供的信息，避免引入无法从附件识别的力学参数。第四问的半径是观测输入，不再另设需要拟合的收缩预测方程；这与根据局部收缩速度预测几何的文献模型 [3] 有所区别。

2 模型假设与符号

2.1 基本假设

(1) 药材为径向各向同性的轴对称连续介质，忽略周向差异和中段轴向梯度，以一维径向场近似中截面分布。

(2) 第一至第三问半径不变；第四问长度不变、截面均匀径向收缩，整体半径历史适用于所研究的中截面。

(3) 初始温度和干基含水率均匀。热物性与扩散率按各问指定附录取局部状态值，不另行拟合参数。

(4) 附件 1、附件 2 均作分段线性插值。4 h 后的烘房环境以最后一小时均值为基准平台，末观测之后用 60 s 过渡到平台；这是长期环境假设。

(5) 第二至第四问未另给表面系数，故沿用第一问的传热系数 $25 \text{ W}/(\text{m}^2\cdot\text{K})$ 和传质系数 $8\times 10^{-7} \text{ m/s}$ 。气相含湿量与固相干基含水率的质量基准不同，在缺少吸附等温线时，将二者之差解释为有效传质推动力。

(6) 忽略蒸发潜热、热湿交叉迁移和力学应力，以经验密度和比热的乘积表示有效体积热容。模型按有效介质解释，其质量与热力学闭合限制见第 8.5 节。

(7) 第四问干物质无损失、初始干物质体积密度均匀；在均匀收缩下，该干密度只随时间变化。此处的守恒干密度与附录经验密度分开定义，二者的相容性在第 8 节检验。

2.2 主要符号

符号	含义	单位
t	自烘干起点计的时间	s
$r, R(t), R_0$	物理半径、当前表面半径、初始表面半径	m
$\xi = r/R(t)$	随材料运动的归一化径向坐标	1
T, T_K	摄氏温度、开尔文温度	°C、K
C	固相干基含水率	kg/kg
T_∞, C_∞	烘房温度、环境含湿量	°C、kg/kg
ρ, c_p, k	经验密度、比热容、导热系数	kg/m ³ 、J/(kg·K)、W/(m·K)
D	有效水分扩散率	m ² /s
h_T, h_m	对流传热、有效传质系数	W/(m ² ·K)、m/s
ρ_d, v_r	当前干物质体积密度、径向骨架速度	kg/m ³ 、m/s
t_*, t_{60}	临界时刻、其后首个严格达标整分钟	s

所有计算距离使用 m，表格按题意显示 cm；内部温度保存为 °C，只在扩散率的 Arrhenius 项中换算为 K。后文的“水分浓度”均指题目定义的干基含水率。

3 数据准备与统一数值方法

3.1 数据核验及环境延拓

附件 1 含 241 组记录，覆盖 0—14400 s，间隔 60 s；附件 2 含 145 组记录，覆盖 0—259200 s，间隔 1800 s。提取后的两份 CSV 与原始工作簿逐值相同。半径从 2.000 cm 降至末段的 1.198 cm；原始观测值保留，不以拟合曲线代替输入。

对相邻观测点，环境或半径变量 f 采用

$$f(t) = f_i + \frac{t-t_i}{t_{i+1}-t_i} (f_{i+1} - f_i), t_i \leq t \leq t_{i+1} \quad (1)$$

附件 1 最后一小时包含 61 个观测点，温度和含湿量均值分别为 49.998934 °C、0.04998754 kg/kg，标准差分别为 0.154353 °C、0.00015013 kg/kg；对应线性趋势约为 0.002824 °C/h 和 -0.00001288 kg/(kg·h)。尾段波动较小，因此选取该时段均值作为长期平台。由于观测仪覆盖前 4 h，第 8.3 节进一步检验延拓方式对终点的影响。

3.2 环形有限体积离散

在径向节点周围构造环形控制体。省略所有单元共有的轴向长度后，控制体体积与外侧面积分别为 $V_i = \pi(r_{i+1/2}^2 - r_{i-1/2}^2)$ 、 $A_{i+1/2} = 2\pi r_{i+1/2}$ 。对一般传输变量 u ，令蓄积系数为 s 、输运系数为 Γ ，半离散方程为

$$s_i V_i \frac{du_i}{dt} = A_{i+1/2} \Gamma_{i+1/2} \frac{u_{i+1} - u_i}{\Delta r} - A_{i-1/2} \Gamma_{i-1/2} \frac{u_i - u_{i-1}}{\Delta r} \quad (2)$$

对热传导取 $u = T, s = \rho c_p, \Gamma = k$ ；对有效水分扩散取 $u = C, s = 1, \Gamma = D$ 。界面系数采用调和平均：

$$\Gamma_{i+1/2} = \frac{2\Gamma_i \Gamma_{i+1}}{\Gamma_i + \Gamma_{i+1}} \quad (3)$$

圆心控制体的内侧面积为零，因此不直接计算 $1/r$ 的奇异表达；表面控制体将外侧项替换为题设 Robin 通量。相邻单元的内部通量大小相等、符号相反，求和后只剩边界通量，便于独立核查离散平衡。

3.3 时间积分、正性与外推

空间离散后得到刚性常微分方程组，采用后向差分公式（BDF）隐式积分。为保持含水率为正，令 $z = \ln C$ ，按 $z_t = C_t/C$ 更新，并由 $C = \exp z$ 恢复含水率。这一变量替换保持原水分方程不变。积分器自适应选取步长，再在题设时刻输出结果。

在共同的物理输出节点上，对间隔数为 N 、 $2N$ 的两级解作二阶 Richardson 外推：

$$u_{\text{ext}} = \frac{4u_{2N} - u_N}{3} \quad (4)$$

第一问使用 5120/10240 区间，第二、三问使用 2560/5120 区间，第四问使用 640/1280 区间。外推以渐近二阶误差为前提，精度由网格加密结果评估，四位小数仅作为题设输出格式。第二问按 600 s 分段求解，每段均传递未舍入的完整节点终态。

4 问题一：固定半径预热模型

4.1 方程、初值与边界

在 $0 < r < R_0 = 0.02$ m 内，采用

$$\begin{aligned} \rho c_p \frac{\partial T}{\partial t} &= \frac{1}{r} \frac{\partial}{\partial r} \left(r k \frac{\partial T}{\partial r} \right) \\ \frac{\partial C}{\partial t} &= \frac{1}{r} \frac{\partial}{\partial r} \left(r D(C) \frac{\partial C}{\partial r} \right) \end{aligned} \quad (5)$$

附录 2 给定 $\rho = 820 \text{ kg/m}^3$ 、 $c_p = 2600 \text{ J/(kg}\cdot\text{K)}$ 、 $k = 0.36 \text{ W/(m}\cdot\text{K)}$ ，水分扩散率为

$$D(C) = 7 \times 10^{-9} \exp\left(-\frac{0.89}{C}\right) \text{ m}^2/\text{s} \quad (6)$$

初始条件为 $T(r, 0) = 28 \text{ }^\circ\text{C}$ 、 $C(r, 0) = 2.55 \text{ kg/kg}$ ；圆心满足 $T_r(0, t) = C_r(0, t) = 0$ 。径向向外取正，表面边界为

$$\begin{aligned} -kT_r(R_0, t) &= h_T[T_s - T_\infty(t)] \\ -DC_r(R_0, t) &= h_m[C_s - C_\infty(t)] \end{aligned} \quad (7)$$

当烘房温度高于表面时，向外热通量为负，热量进入药材；当表面含水率高于有效环境值时，水分向外迁移。第一问热物性为常数、扩散率不依赖温度，因此温度与水分可以分别求解。

4.2 指定位置的计算结果

表 1、表 2 按题目时刻和位置列出结果。完整 1—1800 s、径向间隔 0.1 cm 的两类场见支撑材料 result1.xlsx。所有浓度和温度按题意保留四位小数。

表 1 代表位置温度 $T/^\circ\text{C}$

时间/s	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
100	28.0001	28.0003	28.0040	28.0327	28.1801
300	28.0408	28.0635	28.1514	28.3681	28.8489
600	28.4533	28.5360	28.8039	29.3159	30.1652
900	29.3243	29.4583	29.8755	30.6161	31.7304
1200	30.5427	30.7098	31.2223	32.1126	33.4276
1500	31.9957	32.1867	32.7660	33.7463	35.1203
1800	33.5753	33.7720	34.3642	35.3621	36.7856

表 2 代表位置水分浓度 $C/(\text{kg/kg})$

时间/s	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
100	2.5500	2.5500	2.5500	2.5500	2.2470
300	2.5500	2.5500	2.5500	2.5492	2.0508
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3755	1.5102

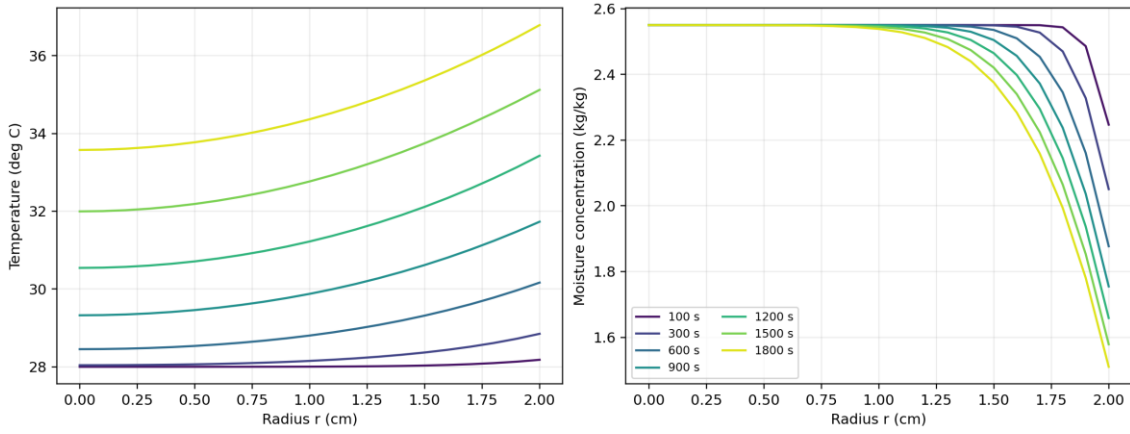


图 1 预热阶段的径向温度与含水率分布；不同曲线对应图例给出的时刻。

由图 1 可见，热量由表面逐渐传向中心，水分损失则主要集中在表层。1800 s 时，表面与中心温差为 3.2103 °C，表面含水率降至 1.5102 kg/kg，中心仍接近初值。热量与水分的迁移速度不同，使预热末期形成了不同程度的径向梯度。

5 问题二：变物性温湿耦合模型

5.1 经验物性与耦合关系

从烘干初始时刻统一采用附录 3 物性 [1]：

$$\rho = 650 + 128C, c_p = 1450 + 2736 \frac{C}{C+1}, k = 0.21 + 0.38 \frac{C}{C+1} \quad (8)$$

$$D(C, T) = 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C}\right) \exp\left(-\frac{3850}{T_K}\right), T_K = T + 273.15 \quad (9)$$

将式 (8) — (9) 代入式 (5)，在题设初值和式 (7) 的边界下联立求解。含水率改变密度、比热和导热系数，温度则通过扩散率影响水分迁移，二者由此构成非线性耦合。离散时保留导热系数在通量散度内的位置，并将 Arrhenius 项中的温度换算为开尔文温度。

水分方程基于固定骨架和空间均匀干密度的有效扩散近似。能量方程中的经验 $\rho(C)$ 用于构造有效体积热容；若将 $\rho(C)/(1+C)$ 同时解释为静止骨架的守恒干密度，两种定义将不再相容。本文保留有效模型的解释，并在第 8.5 节讨论该限制。

5.2 前三小时结果与全程输出

表 3 3 小时内药材温度 (°C)

时间/h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
0.5	32.1892	32.3821	32.9659	33.9608	35.4130
1.0	40.3816	40.5536	41.0605	41.8782	42.9976
1.5	45.8468	45.9348	46.1930	46.6051	47.1400
2.0	48.4502	48.4881	48.5984	48.7736	49.0033
2.5	49.4670	49.4792	49.5136	49.5653	49.6609
3.0	49.8495	49.8553	49.8746	49.9101	49.9664

表 4 3 小时内药材水分浓度 (kg/kg)

时间/h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
0.5	2.5499	2.5489	2.5256	2.3257	1.6486
1.0	2.5257	2.4948	2.3578	2.0230	1.4711
1.5	2.3861	2.3256	2.1344	1.8020	1.3476
2.0	2.1709	2.1084	1.9236	1.6259	1.2311
2.5	1.9566	1.9006	1.7360	1.4720	1.1166
3.0	1.7662	1.7165	1.5702	1.3333	1.0081

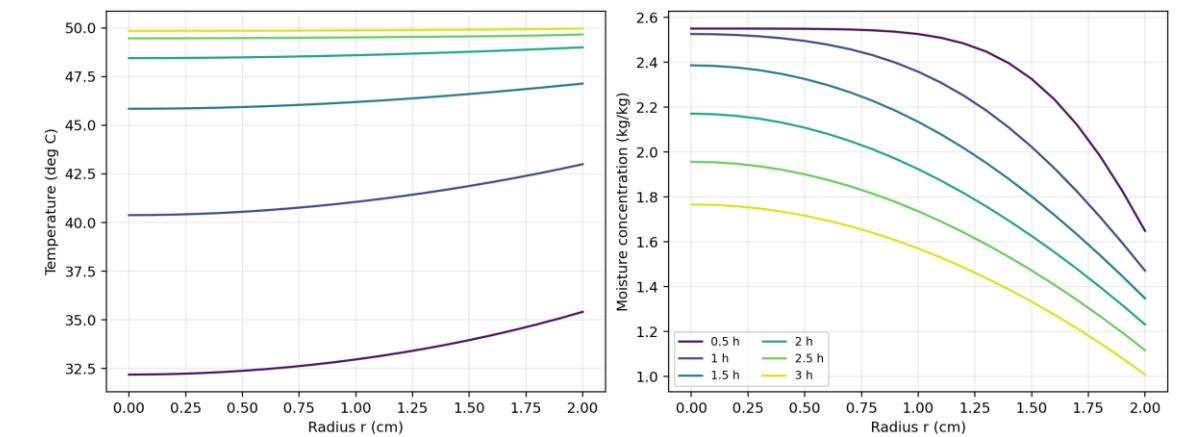


图 2 变物性模型前三小时的径向温湿分布。

3 h 时中心与表面温差仅为 0.1169 °C，但含水率差仍为 0.7581 kg/kg，且各处均未达到 0.15 kg/kg 阈值。升温基本完成后，低含水率区的扩散率继续减小，内部水分向表面的迁移仍需较长时间。这也解释了为什么不能以表面温度稳定作为烘干结束判据。

表 3、表 4 列出前三小时结果；全程温湿场按 1 s 时间间隔和 0.1 cm 径向间隔输出，延伸至第三问首个严格达标整分钟。长期计算采用第 3.1 节的环境边界，前 4 h 最大积分步长为 2 s，此后为 60 s，相对容差为 2×10^{-10} 。输出文件、无损压缩方式及跨问一致性检查见附录 A.2。

6 问题三：固定半径的干燥终点

6.1 全域阈值与严格不等式

沿用第二问方程，定义

$$g(t) = \max_{0 \leq r \leq R_0} C(r, t) - 0.15, t_* = \inf\{t: g(t) < 0\} \quad (10)$$

在终点邻域内，含水率连续下降，临界时刻满足 $g(t_*) = 0$ ，其后才满足严格不等式。计算时遍历所有径向节点取最大值；外推结果显示含水率沿半径向外非增，因而本算例由中心位置控制干燥终点。

在阈值两侧，对各径向位置的未舍入含水率分别插值，并取其中最晚的阈值到达时间。这样可保留最湿位置随时间变化的可能性。若以整分钟为停止时刻，则定义

$$t_{60} = 60 \left(\left\lfloor \frac{t_*}{60} \right\rfloor + 1 \right) \quad (11)$$

6.2 时长与指定结果

第三问临界时刻为 206906.2447 s（57.4740 h），首个严格达标整分钟为 206940 s（57.4833 h）。

表 5 药材烘干过程的水分浓度（kg/kg）

时间/h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
6	1.0170	0.9881	0.9015	0.7550	0.5328
12	0.4561	0.4432	0.4031	0.3297	0.1642
18	0.2988	0.2916	0.2687	0.2253	0.0845
24	0.2378	0.2327	0.2167	0.1855	0.0665
30	0.2058	0.2018	0.1892	0.1641	0.0598
36	0.1859	0.1825	0.1718	0.1504	0.0566
42	0.1721	0.1691	0.1597	0.1407	0.0548
48	0.1618	0.1592	0.1507	0.1334	0.0537
54	0.1539	0.1514	0.1436	0.1277	0.0530
临界终点 57.4740	0.1500	0.1477	0.1402	0.1249	0.0526

表 5 最后一行为临界时刻结果，中心显示值 0.1500 对应阈值边界。严格达标的停止时刻按式（11）另行确定；工作簿同时列出分钟序列、临界行和首个严格达标整分钟行。

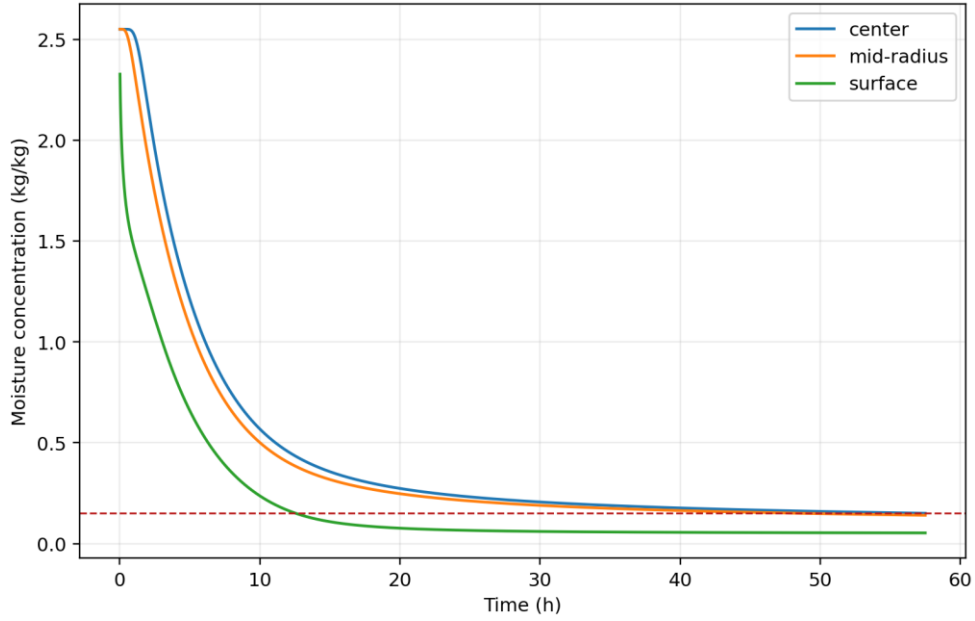


图 3 固定半径下中心、中半径和表面的长期含水率历程；虚线为 0.15 kg/kg 阈值。

图 3 显示，表面含水率先于中心下降，后期各位置的变化均趋缓。干燥终点因此应由全域最大含水率控制，表面或平均值达标不足以保证内部干燥完成。上述时长以所选长期环境平台为条件，其敏感性见第 8.3 节。

7 问题四：收缩移动边界模型

7.1 从干物质与水分守恒出发

附件 2 给出 $R(t)$ ，在均匀径向收缩、长度不变假设下，材料速度为 $v_r = (R/R)r$ 。设当前干物质体积密度为 ρ_d ，相对骨架的水分通量为 $\mathbf{j}_w$ ，则

$$\begin{aligned}\partial_t \rho_d + \nabla \cdot (\rho_d \mathbf{v}) &= 0 \\ \partial_t (\rho_d C) + \nabla \cdot (\rho_d C \mathbf{v} + \mathbf{j}_w) &= 0\end{aligned}\quad (12)$$

采用有效 Fick 通量 $\mathbf{j}_w = -\rho_d D \nabla C$ ，从两式消去干物质连续方程，得

$$\frac{DC}{Dt} = \frac{1}{\rho_d} \nabla \cdot (\rho_d D \nabla C) \quad (13)$$

均匀初始干密度与均匀径向收缩给出 $\rho_d(t)R(t)^2 = \rho_{d0}R_0^2$ ，故干密度在空间上均匀，才可在式 (13) 的散度内外约去，得到

$$\frac{\partial C}{\partial t} + \frac{R}{r} \frac{\partial C}{\partial r} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial C}{\partial r} \right) \quad (14)$$

由于 C 表示单位干物质质量所含的水分，体积变化已由骨架输运与干物质连续方程共同描述，式 (14) 无需再加入体积压缩源项。

7.2 固定域坐标与表面通量

令 $\xi = r/R(t)$ ，则固定 ξ 的网格速度与骨架速度一致，坐标变换项与式 (14) 的对流项相消。在 $0 < \xi < 1$ 内，采用

$$\rho(C)c_p(C)\frac{\partial T}{\partial t}\bigg|_{\xi} = \frac{1}{R(t)^2\xi}\frac{\partial}{\partial \xi}\left(\xi k(C)\frac{\partial T}{\partial \xi}\right) \quad (15)$$

$$\frac{\partial C}{\partial t}\bigg|_{\xi} = \frac{1}{R(t)^2\xi}\frac{\partial}{\partial \xi}\left(\xi D(C,T)\frac{\partial C}{\partial \xi}\right) \quad (16)$$

热方程沿材料坐标采用有效热容近似，不额外计入变形功及相变热。圆心为 $T_{\xi} = C_{\xi} = 0$ ，实际表面的边界条件为

$$\begin{aligned} -\frac{k}{R}T_{\xi}(1,t) &= h_T(T_s - T_{\infty}) \\ -\frac{D}{R}C_{\xi}(1,t) &= h_m(C_s - C_{\infty}) \end{aligned} \quad (17)$$

变换后，内部扩散项含 R^{-2} 尺度，表面通量含 R^{-1} 尺度，收缩由这两处共同进入离散方程。

附录 4 的局部物性为

$$\rho = 760 + 90C, c_p = 1850 + 2150\frac{C}{C+1}, k = 0.12 + 0.20\frac{C}{C+1} \quad (18)$$

$$D(C,T) = 4.2 \times 10^{-4} \exp\left(-\frac{0.30}{C}\right) \exp\left(-\frac{3850}{T_K}\right) \quad (19)$$

7.3 固定物理位置的映射与结果

每个时刻以 $\xi = r/R(t)$ 将题设固定物理位置映射到计算网格。当 $r > R(t)$ 时，该位置已在药材之外，工作簿对应单元格留空；“药材表面”列始终取 $\xi = 1$ ，随实际边界移动。

第四问临界时刻为 183931.0374 s (51.0920 h)，首个严格达标整分钟为 183960 s (51.1000 h)。

表 6 收缩药材烘干过程的水分浓度 (kg/kg)

时间/h	0 cm	0.5 cm	1.0 cm	药材表面
6	1.7196	1.5377	1.0226	0.4207
12	0.7377	0.6546	0.4083	0.1669
18	0.4088	0.3687	0.2397	0.0892
24	0.2851	0.2611	0.1790	0.0673
30	0.2263	0.2094	0.1493	0.0595
36	0.1928	0.1797	0.1317	0.0560
42	0.1712	0.1604	0.1200	0.0541
48	0.1561	0.1468	0.1116	0.0530
烘干临界时刻 51.091955	0.1500	0.1413	0.1081	0.0526

6 h 时半径已为 1.374 cm，之后继续非增；因此表 6 所列时刻的 1.5 cm 和 2 cm 均在实体外，只保留 0、0.5、1 cm 及实际表面。完整 result4.xlsx 仍保留全部 0.1 cm 固定位置与真实表面列，并以空白标记实体外位置。

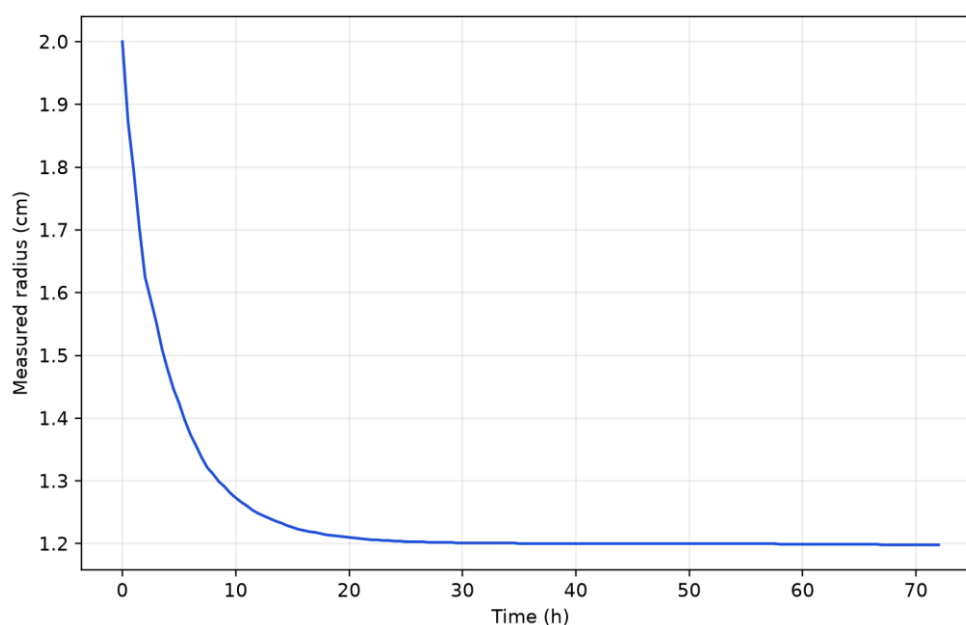


图 4 附件 2 的半径历史及分段线性插值。

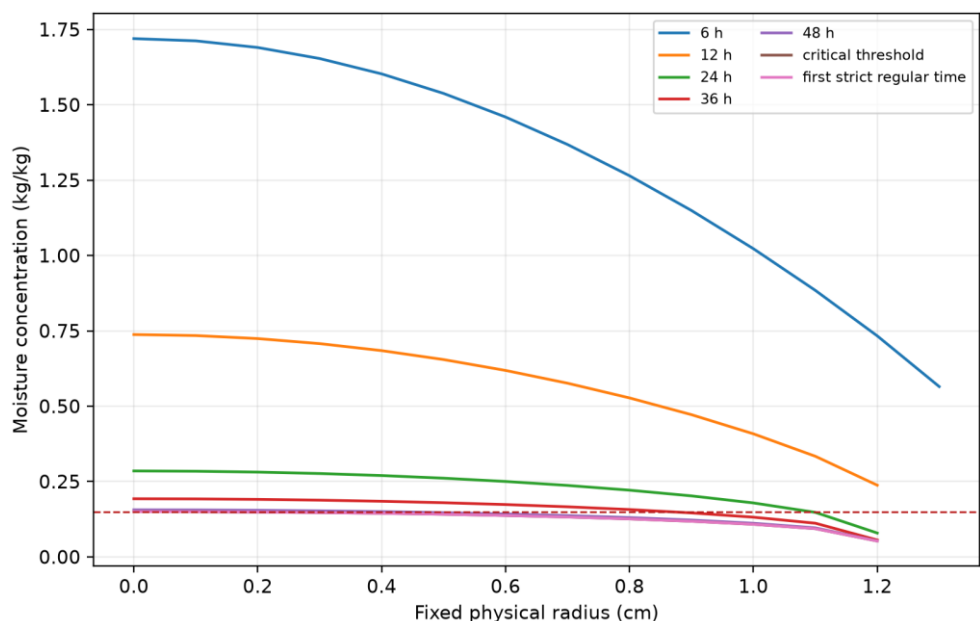


图 5 收缩药材在代表时刻的径向含水率；每条曲线右端是当时实际表面。

第三、四问的临界时长相差约 6.38 h，这一差异同时包含物性与几何变化的影响。为单独考察收缩作用，第 8.4 节保持附录 4 物性不变，仅将半径固定为初始值进行对照。

8 模型检验与灵敏度分析

8.1 网格与程序一致性

第一问两级网格在全部输出节点的最大温度和含水率差分别约为 $1.03 \times 10^{-6} \text{ }^{\circ}\text{C}$ 、 $4.57 \times 10^{-6} \text{ kg/kg}$ ；第二问前三小时对应最大差约为 $8.14 \times 10^{-7} \text{ }^{\circ}\text{C}$ 、 $1.49 \times 10^{-5} \text{ kg/kg}$ 。最终结果采用式（4）的外推值，网格差用于评估数值误差，保留四位小数则服务于题设表格要求。

表 7 两类终点的空间加密结果

模型	径向区间或外推组合	临界时长/h
第三问	1280	57.4881201
第三问	2560	57.4772860
第三问	5120	57.4747882
第三问	2560/5120 外推	57.4739569
第四问	640	51.0996704
第四问	1280	51.0938837
第四问	640/1280 外推	51.0919548

第三、四问相邻外推组合的终点差分别约为 1.02 s、0.98 s。第四问另用单元中心有限体积程序复算，1600/3200 单元外推得到 51.0919474 h，表 6 代表时刻的含水率与

主程序保留四位小数后的值一致。独立计算支持了结果的一致性；由于观测收敛阶仍低于 2，外推差仅作为误差量级参考。

程序同时检验了平衡场保持、边界通量方向、固定半径退化和阈值插值等情形，95 项自动化测试全部通过。表 1—6 与对应数值输出一致；工作簿回读和跨问比较的详细记录列于附录 A.2。

8.2 水分通量平衡

固定半径下，截面平均含水率满足

$$\overline{C}(t) - \overline{C}(0) = -\frac{2h_m}{R_0} \int_0^t [C_s(\tau) - C_\infty(\tau)] d\tau \tag{20}$$

移动域中定义 $\overline{C} = 2 \int_0^1 C\xi \, d\xi$ ，由式（16）—（17）得

$$\frac{d\overline{C}}{dt} = -\frac{2h_m}{R(t)} [C_s(t) - C_\infty(t)] \tag{21}$$

以前 600 s 的逐秒输出和后续分钟输出积分边界通量，第三问两级网格的相对最大残差约为 1.21×10^{-6} ，第四问分别约为 1.61×10^{-6} 、 1.69×10^{-6} 。残差较小，说明离散解与模型自身的水分通量平衡关系一致。

8.3 长期环境与表面系数情景

表 8 第三问长期平台敏感性（同为 640 区间）

4 h 后环境设定	临界时长/h
最后一小时均值	57.5413
最后半小时均值	57.5151
最后观测值保持	57.2375
温度降低、含湿量升高各一个尾段标准差	57.8259
温度升高、含湿量降低各一个尾段标准差	57.2587

表 8 各情景采用相同网格，临界时长的跨度约为 0.59 h，明显大于相邻网格外推的差异。该范围表示指定环境设定下的预测变化，并非统计置信区间；比较时均以表内基准为参照。

表 9 第四问表面系数敏感性（同为 320 区间）

变化系数	基准的 0.8 倍/h	基准/h	基准的 1.2 倍/h
传质系数	51.8009	51.1236	50.7400
传热系数	51.1418	51.1236	51.1117

在表 9 的取值范围内，传质系数变化引起的终点偏移大于传热系数变化的影响。这与后期主要受水分迁移制约的结果相符。0.8 倍和 1.2 倍为指定扰动幅度，实际参数的不确定性仍需实验标定。

8.4 收缩处理与固定几何对照

保持附件 2 全部观测点不变，将线性插值替换为单调 PCHIP，在相同 320 区间下终点提前约 12.603 s。另将内部半径观测时刻统一提前或推后 15 min，终点相对同组基准改变约 -0.135 h、+0.137 h。前者比较插值方法，后者改变观测时刻，二者不是同一种检验。

使用附录 4 相同物性、但固定 $R = R_0$ 的对照模型，640/1280 区间外推终点为 129.8481 h；实测收缩模型约为 51.0920 h。半径减小通过式（16）的扩散尺度与式（17）的表面尺度共同影响水分迁移。该对照隔离了所选模型中的几何差异，但并非对真实药材收缩效应的独立实验测量。

8.5 密度与收缩的相容性

如果把附录 4 的 $\rho(C)$ 强制解释为当前真实湿基密度，则应有 $\rho_d = \rho(C)/(1 + C)$ ，相应总干物质质量为

$$M_d(t) = 2\pi LR(t)^2 \int_0^1 \frac{760+90C(\xi,t)}{1+C(\xi,t)} \xi d\xi \quad (22)$$

对非均匀含水率场积分，在 1280 区间自身临界时刻得到 $M_d(t_*)/M_d(0) \approx 0.890289$ ，隐含偏差约为 10.97%。因此，若将给定密度解释为真实湿密度，长度不变的实测收缩与干物质守恒在当前解中无法同时严格满足。该偏差用于诊断模型闭合关系，不代表实测干物质损失。

本文将附录密度用于经验有效体积热容，水分方程中的干密度则由均匀收缩假设定义。两者分开使用给出了有效模型的适用解释，但密度与几何关系的不相容仍是其结构限制。要建立严格的多组分守恒模型，还需补充孔隙率、局部变形或长度变化等信息，现有附件不足以确定这些关系。

9 模型评价与结论

9.1 模型特点

本文以统一的有限体积离散处理常物性、变物性和实测收缩三种情形，得到题设位置的温湿分布，并以全域最大含水率确定干燥终点。移动坐标使变化的几何区域转为固定计算区间，同时保留实际表面与固定物理位置的区别。网格加密、通量平衡及独立程序比较为数值结果提供了相互补充的检查。

9.2 局限与改进方向

预测时长主要受模型结构与输入解释约束。一维中截面近似尚未量化端面影响，长期环境依赖前 4 h 观测的延拓，气固边界采用有效浓度差并沿用题设表面系数；模型还忽略了潜热，且经验密度与收缩关系未严格闭合。现有检验支持数值求解的一致性，物理预测精度仍需独立内部含水率观测验证。

后续应优先补充长期烘房记录与内部温湿测量，检验环境平台和干燥终点；再通过二维模型、吸附平衡关系及几何变形观测，评估端面效应和质量闭合对预测的影响。

9.3 结论

四问结果共同表明，药材烘干先经历明显的表层失水，随后进入内部水分缓慢迁移的阶段；温度接近均匀时，含水率仍可能远高于达标阈值。在本文假设下，固定半径与实测收缩模型的临界时长分别为 57.4740 h、51.0920 h，若按整分钟停止则分别取 57.4833 h、51.1000 h。采用相同物性的固定几何对照进一步说明，收缩是影响预测时长的重要因素，实际工艺应用仍需结合长期环境和内部含水率测量校验。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于模型推导辅助、代码实现与调试、数值复算和结果核查、论文初稿组织及语言润色，详细使用情况见支撑材料。

参赛队对模型、代码、数值、参考文献和最终表述承担核验责任。工具的具体用途与核验方式见支撑材料中的 AI 工具使用详情，本节披露不作为参考文献引用。

参考文献

- [1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 A 题：药材的烘干问题及附件[Z]. 2026.
- [2] DA SILVA W P, E SILVA C M D P S, GAMA F J A. Estimation of thermo-physical properties of products with cylindrical shape during drying: The coupling between mass and heat[J]. Journal of Food Engineering, 2014, 141: 65-73. DOI: 10.1016/j.jfoodeng.2014.05.010.
- [3] ADROVER A, VENDITTI C, BRASIELLO A. A non-isothermal moving-boundary model for continuous and intermittent drying of pears[J]. Foods, 2020, 9(11): 1577. DOI: 10.3390/foods9111577.

附录 A 支撑材料与复现说明

支撑材料 ZIP 内的文件列表如下。论文电子版从摘要页开始，不含承诺书和编号专用页。文中四位小数用于题设输出，终点判定始终使用未舍入结果。

```
AI 工具使用详情.pdf
README_复现说明.md
attachment1.csv
attachment2.csv
final_all_questions_audit.json
flat_support_audit.json
problem1.py
problem1_radial_profiles.png
problem1_time_histories.png
problem2.py
problem2_full_process_audit.json
problem2_radial_profiles.png
problem2_time_histories.png
problem3.py
problem3_radial_profiles.png
problem3_time_histories.png
problem4.py
problem4_independent.py
problem4_independent_validation.json
problem4_production_diagnostics.json
problem4_radial_profiles.png
problem4_radius_history.png
problem4_review_experiments.json
problem4_time_histories.png
problem4_workbook_audit.json
requirements.txt
restore_full_result2.py
result1.xlsx
result2.xlsx
result2_full_process.npz
result3.xlsx
result4.xlsx
run_problem1.py
run_problem2.py
run_problem3.py
run_problem4.py
verify_problem4.py
verify_problem4_independent.py
```

A.1 运行顺序

先在支撑材料根目录安装 requirements.txt 中的依赖。Python 版本须为 3.11 或以上。支撑材料采用单层结构，按下面顺序运行即可：

```
python -m pip install -r requirements.txt
python run_problem1.py
python run_problem2.py
python run_problem3.py
python run_problem4.py
```

四个求解脚本在根目录产生 problem1_result.json 至 problem4_result.json，并生成相应图形；不会覆盖已填写的四份结果工作簿。复现所需 CSV 已包含在支撑材料中。

若仅需查看全程第二问 Excel，无须重新求解，在支撑材料根目录执行 python restore_full_result2.py，即可由压缩数组还原 result2_full_process.xlsx。

A.2 全程输出与数值核验记录

第二问全程逐秒 Excel 约 27.7 MB。为满足支撑材料压缩包不超过 20 MB 的要求，包中以 result2_full_process.npz 无损保存四位小数交付值，并附 Excel 恢复脚本；result2.xlsx 为原前三小时文件。包外另提供的全程 Excel 与恢复文件的两张工作表 XML 逐字节相同。

全程积分沿用第二问的方程、初值和完整节点终态，输出不由舍入后的分钟值插值得到。其前 3 h 与原第二问交付值完全一致，与第三问独立分钟积分的最大四位小数差为 0.0001 kg/kg，差异来自积分分段与舍入边界。

原四份工作簿共回读 700974 个单元格，题设表 1—6 与对应数值输出一致；第二、三问原始分钟序列在前三小时重叠的 3780 个含水率值相同。第二问全程逐秒工作簿另有独立回读记录，见 problem2_full_process_audit.json。

支撑材料根目录保存原四问审计、第四问敏感性和密度诊断、全程逐秒结果审计。程序核验与参赛队人工审查是两类不同记录；AI 工具使用详情仅记录可确认的自动检查，不代替或虚构人工核验。

附录 B 完整源程序

下列代码为本次建模、输出与验证所需源文件全文。支撑材料中的程序均直接位于根目录，文件名和代码缩进保持不变。附录页数不计入正文限制。

problem1.py

```
"""Problem 1 material laws and chamber boundary interpolation."""

from __future__ import annotations

import csv
from dataclasses import dataclass
from pathlib import Path

import numpy as np
from numpy.typing import ArrayLike, NDArray
from scipy.integrate import solve_ivp
from scipy.sparse import diags

@dataclass(frozen=True)
class ChamberHistory:
    """Observed chamber boundary conditions."""

    time_s: NDArray[np.float64]
    temperature_c: NDArray[np.float64]
    moisture_concentration: NDArray[np.float64]

    def __post_init__(self) -> None:
        time = np.asarray(self.time_s, dtype=float)
        temperature = np.asarray(self.temperature_c, dtype=float)
        moisture = np.asarray(self.moisture_concentration, dtype=float)
        if time.ndim != 1 or temperature.shape != time.shape or moisture.shape != time.shape:
            raise ValueError("Chamber history columns must be one-dimensional and equal-sized")
        if any(np.any(~np.isfinite(values)) for values in (time, temperature, moisture)):
            raise ValueError("Chamber history must contain only finite values")
```

```

        if time.size < 2 or np.any(np.diff(time) <= 0.0):
            raise ValueError("Chamber times must be strictly increasing")
        if np.any(moisture < 0.0):
            raise ValueError("Chamber moisture concentration must be non-negative")
        object.__setattr__(self, "time_s", time)
        object.__setattr__(self, "temperature_c", temperature)
        object.__setattr__(self, "moisture_concentration", moisture)

@dataclass(frozen=True)
class Problem1Parameters:
    radius_m: float = 0.02
    initial_temperature_c: float = 28.0
    initial_moisture_concentration: float = 2.55
    density_kg_m3: float = 820.0
    heat_capacity_j_kg_k: float = 2600.0
    thermal_conductivity_w_m_k: float = 0.36
    heat_transfer_coefficient_w_m2_k: float = 25.0
    mass_transfer_coefficient_m_s: float = 8.0e-7

@dataclass(frozen=True)
class Problem1Solution:
    time_s: NDArray[np.float64]
    radius_m: NDArray[np.float64]
    temperature_c: NDArray[np.float64]
    moisture_concentration: NDArray[np.float64]

def load_chamber_history_csv(path: str | Path) -> ChamberHistory:
    """Load the three columns supplied in Attachment 1 from a UTF-8 CSV."""
    rows: list[tuple[float, float, float]] = []
    with Path(path).open("r", encoding="utf-8-sig", newline="") as handle:
        reader = csv.DictReader(handle)
        required = {"时间", "温度", "水分浓度"}
        if reader.fieldnames is None or not required.issubset(reader.fieldnames):
            raise ValueError("Attachment 1 must contain 时间, 温度, 水分浓度 columns")
        for line_number, row in enumerate(reader, start=2):
            try:
                rows.append((float(row["时间"]), float(row["温度"]), float(row["水分浓度"])))
            except (TypeError, ValueError) as exc:
                raise ValueError(f"Invalid numeric value on CSV line {line_number}") from exc
    if not rows:
        raise ValueError("Attachment 1 contains no data rows")
    data = np.asarray(rows, dtype=float)
    if not np.all(np.isfinite(data)):
        raise ValueError("Attachment 1 contains non-finite values")
    return ChamberHistory(data[:, 0], data[:, 1], data[:, 2])

def diffusivity_q1(concentration: ArrayLike) -> NDArray[np.float64]:
    """Return  $D=7e-9 \exp(-0.89/C)$  in square metres per second."""
    values = np.asarray(concentration, dtype=float)
    if np.any(values <= 0.0):
        raise ValueError("Moisture concentration must be positive")
    return 7.0e-9 * np.exp(-0.89 / values)

def interpolate_history(
    query_time_s: float,
    time_s: ArrayLike,
    values: ArrayLike,
) -> float:
    """Linearly interpolate an observed chamber history."""
    return float(np.interp(query_time_s, time_s, values))

def nodal_control_volumes(node_count: int, radius_m: float) -> NDArray[np.float64]:
    """Return annular control volumes per unit cylinder length for nodal radii."""
    if node_count < 2:

```

```

        raise ValueError("At least two radial nodes are required")
    if radius_m <= 0.0:
        raise ValueError("Cylinder radius must be positive")
    spacing = radius_m / (node_count - 1)
    radii = np.linspace(0.0, radius_m, node_count)
    inner = np.maximum(radii - 0.5 * spacing, 0.0)
    outer = np.minimum(radii + 0.5 * spacing, radius_m)
    return np.pi * (outer**2 - inner**2)

def radial_flux_divergence(
    state: ArrayLike,
    coefficient: ArrayLike,
    radius_m: float,
    exchange_coefficient: float,
    ambient_value: float,
) -> NDArray[np.float64]:
    """Conservative cylindrical divergence with a convective outer boundary.

    ``coefficient`` is k for heat or D for moisture. At the outer surface the
    imposed flux is ``-exchange_coefficient * (surface - ambient)``.
    """
    values = np.asarray(state, dtype=float)
    conductance = np.asarray(coefficient, dtype=float)
    if values.ndim != 1 or conductance.shape != values.shape:
        raise ValueError("State and coefficient must be one-dimensional and equal-sized")
    if np.any(conductance < 0.0) or exchange_coefficient < 0.0:
        raise ValueError("Transport coefficients must be non-negative")

    node_count = values.size
    spacing = radius_m / (node_count - 1)
    face_radii = (np.arange(node_count - 1, dtype=float) + 0.5) * spacing
    denominator = conductance[:-1] + conductance[1:]
    face_coefficient = np.divide(
        2.0 * conductance[:-1] * conductance[1:],
        denominator,
        out=np.zeros_like(denominator),
        where=denominator > 0.0,
    )
    face_gradient = np.diff(values) / spacing
    face_rate = 2.0 * np.pi * face_radii * face_coefficient * face_gradient

    integrated_rate = np.zeros_like(values)
    integrated_rate[:-1] += face_rate
    integrated_rate[1:] -= face_rate
    surface_flux = -exchange_coefficient * (values[-1] - ambient_value)
    integrated_rate[-1] += 2.0 * np.pi * radius_m * surface_flux

    return integrated_rate / nodal_control_volumes(node_count, radius_m)

def solve_problem1(
    history: ChamberHistory,
    radial_intervals: int,
    output_times_s: ArrayLike,
    *,
    parameters: Problem1Parameters = Problem1Parameters(),
    relative_tolerance: float = 1.0e-8,
    max_step_s: float = 5.0,
) -> Problem1Solution:
    """Solve the fixed-radius preheating model at requested output times."""
    if radial_intervals < 2:
        raise ValueError("At least two radial intervals are required")
    output_times = np.asarray(output_times_s, dtype=float)
    if output_times.ndim != 1 or output_times.size == 0:
        raise ValueError("Output times must be a non-empty one-dimensional array")
    if np.any(np.diff(output_times) <= 0.0):
        raise ValueError("Output times must be strictly increasing")
    if output_times[0] < history.time_s[0] or output_times[-1] > history.time_s[-1]:
        raise ValueError("Output times must lie within the chamber history")

```

```

node_count = radial_intervals + 1
radii = np.linspace(0.0, parameters.radius_m, node_count)
sparsity = diags(
    [np.ones(node_count - 1), np.ones(node_count), np.ones(node_count - 1)],
    offsets=[-1, 0, 1],
    shape=(node_count, node_count),
    format="csr",
)

thermal_coefficient = np.full(node_count, parameters.thermal_conductivity_w_m_k)

def temperature_rhs(time_s: float, temperature_c: NDArray[np.float64]) -> NDArray[np.float64]:
    ambient = interpolate_history(time_s, history.time_s, history.temperature_c)
    divergence = radial_flux_divergence(
        temperature_c,
        thermal_coefficient,
        parameters.radius_m,
        parameters.heat_transfer_coefficient_w_m2_k,
        ambient,
    )
    return divergence / (parameters.density_kg_m3 * parameters.heat_capacity_j_kg_k)

def log_moisture_rhs(time_s: float, log_concentration: NDArray[np.float64]) -> NDArray[np.float64]:
    concentration = np.exp(log_concentration)
    ambient = interpolate_history(time_s, history.time_s, history.moisture_concentration)
    divergence = radial_flux_divergence(
        concentration,
        diffusivity_q1(concentration),
        parameters.radius_m,
        parameters.mass_transfer_coefficient_m_s,
        ambient,
    )
    return divergence / concentration

time_span = (float(history.time_s[0]), float(output_times[-1]))
temperature_result = solve_ivp(
    temperature_rhs,
    time_span,
    np.full(node_count, parameters.initial_temperature_c),
    method="BDF",
    t_eval=output_times,
    rtol=relative_tolerance,
    atol=1.0e-9,
    max_step=max_step_s,
    jac_sparsity=sparsity,
)
moisture_result = solve_ivp(
    log_moisture_rhs,
    time_span,
    np.full(node_count, np.log(parameters.initial_moisture_concentration)),
    method="BDF",
    t_eval=output_times,
    rtol=relative_tolerance,
    atol=1.0e-10,
    max_step=max_step_s,
    jac_sparsity=sparsity,
)
if not temperature_result.success:
    raise RuntimeError(f"Temperature solver failed: {temperature_result.message}")
if not moisture_result.success:
    raise RuntimeError(f"Moisture solver failed: {moisture_result.message}")

return Problem1Solution(
    time_s=output_times,
    radius_m=radii,
    temperature_c=temperature_result.y.T,
    moisture_concentration=np.exp(moisture_result.y.T),
)

```

```

def sample_solution(solution: Problem1Solution, radius_cm: ArrayLike) -> Problem1Solution:
    """Interpolate a solution to requested physical radii in centimetres."""
    requested_radius_m = np.asarray(radius_cm, dtype=float) * 0.01
    if requested_radius_m.ndim != 1 or requested_radius_m.size == 0:
        raise ValueError("Requested radii must be a non-empty one-dimensional array")
    if np.any(np.diff(requested_radius_m) <= 0.0):
        raise ValueError("Requested radii must be strictly increasing")
    if requested_radius_m[0] < 0.0 or requested_radius_m[-1] > solution.radius_m[-1]:
        raise ValueError("Requested radii lie outside the cylinder")

    temperature = np.vstack(
        [np.interp(requested_radius_m, solution.radius_m, row) for row in solution.temperature_c]
    )
    moisture = np.vstack(
        [
            np.interp(requested_radius_m, solution.radius_m, row)
            for row in solution.moisture_concentration
        ]
    )
    return Problem1Solution(
        time_s=solution.time_s.copy(),
        radius_m=requested_radius_m,
        temperature_c=temperature,
        moisture_concentration=moisture,
    )

def richardson_extrapolate_solutions(
    coarse: Problem1Solution,
    fine: Problem1Solution,
    *,
    order: int = 2,
) -> Problem1Solution:
    """Extrapolate two solutions whose spatial mesh widths differ by a factor of two."""
    if order <= 0:
        raise ValueError("Richardson order must be positive")
    if (
        coarse.temperature_c.shape != fine.temperature_c.shape
        or coarse.moisture_concentration.shape
        != fine.moisture_concentration.shape
    ):
        raise ValueError("Solutions must have matching field shapes")
    if not np.array_equal(coarse.time_s, fine.time_s) or not np.array_equal(
        coarse.radius_m,
        fine.radius_m,
    ):
        raise ValueError("Solutions must use identical output times and radii")
    denominator = 2.0*order - 1.0
    return Problem1Solution(
        time_s=fine.time_s.copy(),
        radius_m=fine.radius_m.copy(),
        temperature_c=fine.temperature_c
        + (fine.temperature_c - coarse.temperature_c) / denominator,
        moisture_concentration=fine.moisture_concentration
        + (fine.moisture_concentration - coarse.moisture_concentration) / denominator,
    )

def result_payload(solution: Problem1Solution) -> dict[str, list]:
    """Convert a sampled solution to the numeric schema used by result1.xlsx."""
    if np.any(~np.isfinite(solution.time_s)) or not np.allclose(solution.time_s,
np.round(solution.time_s), atol=1.0e-10, rtol=0.0):
        raise ValueError("Workbook output times must be whole seconds")
    return {
        "time_s": np.round(solution.time_s).astype(int).tolist(),
        "radius_cm": np.round(solution.radius_m * 100.0, 10).tolist(),
        "temperature_c": np.round(solution.temperature_c, 4).tolist(),
        "moisture_concentration": np.round(solution.moisture_concentration, 4).tolist(),
    }

```

problem2.py

```
"""Coupled variable-property model for Problem 2."""

from __future__ import annotations

from collections.abc import Callable

import numpy as np
from numpy.typing import ArrayLike, NDArray
from scipy.integrate import solve_ivp
from scipy.sparse import bmat, diags

from problem1 import (
    ChamberHistory,
    Problem1Parameters,
    Problem1Solution,
    interpolate_history,
    radial_flux_divergence,
    sample_solution,
)

ScalarLaw = Callable[[NDArray[np.float64]], NDArray[np.float64]]
CoupledLaw = Callable[
    [NDArray[np.float64], NDArray[np.float64]],
    NDArray[np.float64],
]

def _concentration_array(concentration: ArrayLike) -> NDArray[np.float64]:
    values = np.asarray(concentration, dtype=float)
    if np.any(~np.isfinite(values)) or np.any(values < 0.0):
        raise ValueError("Moisture concentration must be finite and non-negative")
    return values

def density_q2(concentration: ArrayLike) -> NDArray[np.float64]:
    """Return Appendix 3 density in kilograms per cubic metre."""
    values = _concentration_array(concentration)
    return 650.0 + 128.0 * values

def heat_capacity_q2(concentration: ArrayLike) -> NDArray[np.float64]:
    """Return Appendix 3 specific heat capacity in joules per kilogram-kelvin."""
    values = _concentration_array(concentration)
    return 1450.0 + 2736.0 * values / (values + 1.0)

def thermal_conductivity_q2(concentration: ArrayLike) -> NDArray[np.float64]:
    """Return Appendix 3 thermal conductivity in watts per metre-kelvin."""
    values = _concentration_array(concentration)
    return 0.21 + 0.38 * values / (values + 1.0)

def diffusivity_q2(
    concentration: ArrayLike,
    temperature_c: ArrayLike,
) -> NDArray[np.float64]:
    """Return Appendix 3 moisture diffusivity using absolute temperature."""
    moisture = _concentration_array(concentration)
    temperature_k = np.asarray(temperature_c, dtype=float) + 273.15
    if np.any(~np.isfinite(temperature_k)) or np.any(temperature_k <= 0.0):
        raise ValueError("Temperature must be finite and above absolute zero")
    if np.any(moisture <= 0.0):
        raise ValueError("Moisture concentration must be positive for diffusivity")
    moisture, temperature_k = np.broadcast_arrays(moisture, temperature_k)
    return 2.4e-3 * np.exp(-0.45 / moisture) * np.exp(-3850.0 / temperature_k)
```

```

def solve_variable_property_fixed_cylinder(
    history: ChamberHistory,
    radial_intervals: int,
    output_times_s: ArrayLike,
    *,
    density_law: ScalarLaw,
    heat_capacity_law: ScalarLaw,
    conductivity_law: ScalarLaw,
    diffusivity_law: CoupledLaw,
    parameters: Problem1Parameters = Problem1Parameters(),
    relative_tolerance: float = 1.0e-8,
    max_step_s: float = 5.0,
    maximum_moisture_threshold: float | None = None,
    initial_time_s: float | None = None,
    initial_temperature_c: ArrayLike | None = None,
    initial_moisture_concentration: ArrayLike | None = None,
) -> Problem1Solution:
    """Solve coupled heat and moisture transport on a fixed cylindrical radius."""
    if radial_intervals < 2:
        raise ValueError("At least two radial intervals are required")
    output_times = np.asarray(output_times_s, dtype=float)
    if output_times.ndim != 1 or output_times.size == 0:
        raise ValueError("Output times must be a non-empty one-dimensional array")
    if np.any(np.diff(output_times) <= 0.0):
        raise ValueError("Output times must be strictly increasing")
    start_time = history.time_s[0] if initial_time_s is None else float(initial_time_s)
    if (
        not np.isfinite(start_time)
        or start_time < history.time_s[0]
        or start_time > output_times[0]
        or output_times[-1] > history.time_s[-1]
    ):
        raise ValueError("Output times must lie within the chamber history")
    if maximum_moisture_threshold is not None and (
        not np.isfinite(maximum_moisture_threshold)
        or maximum_moisture_threshold <= 0.0
        or maximum_moisture_threshold >= parameters.initial_moisture_concentration
    ):
        raise ValueError("Maximum moisture threshold must lie between zero and the initial value")

    node_count = radial_intervals + 1
    radii = np.linspace(0.0, parameters.radius_m, node_count)
    tridiagonal = diags(
        [np.ones(node_count - 1), np.ones(node_count), np.ones(node_count - 1)],
        offsets=[-1, 0, 1],
        shape=(node_count, node_count),
        format="csr",
    )
    sparsity = bmat(
        [[tridiagonal, tridiagonal], [tridiagonal, tridiagonal]],
        format="csr",
    )

    def coupled_rhs(time_s: float, state: NDArray[np.float64]) -> NDArray[np.float64]:
        temperature_c = state[:node_count]
        concentration = np.exp(state[node_count:])
        density = np.asarray(density_law(concentration), dtype=float)
        heat_capacity = np.asarray(heat_capacity_law(concentration), dtype=float)
        conductivity = np.asarray(conductivity_law(concentration), dtype=float)
        diffusivity = np.asarray(
            diffusivity_law(concentration, temperature_c),
            dtype=float,
        )
        expected_shape = concentration.shape
        for name, values in (
            ("density", density),
            ("heat capacity", heat_capacity),
            ("thermal conductivity", conductivity),

```

```

        ("diffusivity", diffusivity),
    ):
        if values.shape != expected_shape or np.any(~np.isfinite(values)) or np.any(values <= 0.0):
            raise ValueError(f"{name} law must return finite positive nodal values")

    ambient_temperature = interpolate_history(
        time_s,
        history.time_s,
        history.temperature_c,
    )
    temperature_rate = radial_flux_divergence(
        temperature_c,
        conductivity,
        parameters.radius_m,
        parameters.heat_transfer_coefficient_w_m2_k,
        ambient_temperature,
    ) / (density * heat_capacity)

    ambient_moisture = interpolate_history(
        time_s,
        history.time_s,
        history.moisture_concentration,
    )
    concentration_rate = radial_flux_divergence(
        concentration,
        diffusivity,
        parameters.radius_m,
        parameters.mass_transfer_coefficient_m_s,
        ambient_moisture,
    )
    log_concentration_rate = concentration_rate / concentration
    return np.concatenate([temperature_rate, log_concentration_rate])

supplied_initial_fields = (
    initial_temperature_c is not None,
    initial_moisture_concentration is not None,
)
if any(supplied_initial_fields) and not all(supplied_initial_fields):
    raise ValueError("Both initial fields must be supplied together")
if all(supplied_initial_fields):
    temperature_initial = np.asarray(initial_temperature_c, dtype=float)
    moisture_initial = np.asarray(initial_moisture_concentration, dtype=float)
    if (
        temperature_initial.shape != (node_count,)
        or moisture_initial.shape != (node_count,)
        or np.any(~np.isfinite(temperature_initial))
        or np.any(~np.isfinite(moisture_initial))
        or np.any(moisture_initial <= 0.0)
    ):
        raise ValueError("Initial fields must be finite positive nodal arrays")
else:
    temperature_initial = np.full(node_count, parameters.initial_temperature_c)
    moisture_initial = np.full(
        node_count,
        parameters.initial_moisture_concentration,
    )
initial_state = np.concatenate(
    [temperature_initial, np.log(moisture_initial)]
)
absolute_tolerance = np.concatenate(
    [np.full(node_count, 1.0e-9), np.full(node_count, 1.0e-10)]
)

threshold_event = None
if maximum_moisture_threshold is not None:

    def threshold_event(time_s: float, state: NDArray[np.float64]) -> float:
        del time_s
        return float(
            np.max(np.exp(state[node_count:])) - maximum_moisture_threshold

```

```

        )

        threshold_event.terminal = True
        threshold_event.direction = -1.0

    result = solve_ivp(
        coupled_rhs,
        (start_time, float(output_times[-1])),
        initial_state,
        method="BDF",
        t_eval=output_times,
        rtol=relative_tolerance,
        atol=absolute_tolerance,
        max_step=max_step_s,
        jac_sparsity=sparsity,
        events=threshold_event,
    )
    if not result.success:
        raise RuntimeError(f"Coupled solver failed: {result.message}")
    result_time = result.t
    result_state = result.y
    if maximum_moisture_threshold is not None:
        if not result.t_events or result.t_events[0].size == 0:
            raise RuntimeError("Maximum moisture threshold was not reached")
        event_time = float(result.t_events[0][0])
        event_state = result.y_events[0][0]
        if result_time.size and np.isclose(
            result_time[-1],
            event_time,
            rtol=0.0,
            atol=1.0e-9,
        ):
            result_time = result_time.copy()
            result_state = result_state.copy()
            result_time[-1] = event_time
            result_state[:, -1] = event_state
        else:
            result_time = np.append(result_time, event_time)
            result_state = np.column_stack([result_state, event_state])
    return Problem1Solution(
        time_s=result_time,
        radius_m=radai,
        temperature_c=result_state[:node_count].T,
        moisture_concentration=np.exp(result_state[node_count:].T),
    )

def solve_problem2(
    history: ChamberHistory,
    radial_intervals: int,
    output_times_s: ArrayLike,
    *,
    parameters: Problem1Parameters = Problem1Parameters(),
    relative_tolerance: float = 1.0e-8,
    max_step_s: float = 5.0,
    maximum_moisture_threshold: float | None = None,
    initial_time_s: float | None = None,
    initial_temperature_c: ArrayLike | None = None,
    initial_moisture_concentration: ArrayLike | None = None,
) -> Problem1Solution:
    """Solve Problem 2 with the four Appendix 3 material-property laws."""
    return solve_variable_property_fixed_cylinder(
        history,
        radial_intervals,
        output_times_s,
        density_law=density_q2,
        heat_capacity_law=heat_capacity_q2,
        conductivity_law=thermal_conductivity_q2,
        diffusivity_law=diffusivity_q2,
        parameters=parameters,
    )

```

```

        relative_tolerance=relative_tolerance,
        max_step_s=max_step_s,
        maximum_moisture_threshold=maximum_moisture_threshold,
        initial_time_s=initial_time_s,
        initial_temperature_c=initial_temperature_c,
        initial_moisture_concentration=initial_moisture_concentration,
    )

def solve_problem2_sampled_in_chunks(
    history: ChamberHistory,
    radial_intervals: int,
    output_times_s: ArrayLike,
    sample_radius_cm: ArrayLike,
    *,
    chunk_duration_s: float = 600.0,
    parameters: Problem1Parameters = Problem1Parameters(),
    relative_tolerance: float = 1.0e-8,
    max_step_s: float = 5.0,
) -> Problem1Solution:
    """Solve Problem 2 in bounded-memory chunks and retain sampled radii only."""
    output_times = np.asarray(output_times_s, dtype=float)
    if output_times.ndim != 1 or output_times.size == 0:
        raise ValueError("Output times must be a non-empty one-dimensional array")
    if np.any(~np.isfinite(output_times)) or np.any(np.diff(output_times) <= 0.0):
        raise ValueError("Output times must be finite and strictly increasing")
    if (
        output_times[0] < history.time_s[0]
        or output_times[-1] > history.time_s[-1]
    ):
        raise ValueError("Output times must lie within the chamber history")
    if not np.isfinite(chunk_duration_s) or chunk_duration_s <= 0.0:
        raise ValueError("Chunk duration must be finite and positive")

    requested_radius_cm = np.asarray(sample_radius_cm, dtype=float)
    node_count = radial_intervals + 1
    nodal_radius = np.linspace(0.0, parameters.radius_m, node_count)
    initial_solution = Problem1Solution(
        time_s=np.array([history.time_s[0]]),
        radius_m=nodal_radius,
        temperature_c=np.full((1, node_count), parameters.initial_temperature_c),
        moisture_concentration=np.full(
            (1, node_count),
            parameters.initial_moisture_concentration,
        ),
    )
    sampled_initial = sample_solution(initial_solution, requested_radius_cm)
    sampled_temperature = np.empty(
        (output_times.size, sampled_initial.radius_m.size),
        dtype=float,
    )
    sampled_moisture = np.empty_like(sampled_temperature)

    next_output_index = 0
    current_time = float(history.time_s[0])
    current_temperature: NDArray[np.float64] | None = None
    current_moisture: NDArray[np.float64] | None = None
    if np.isclose(output_times[0], current_time, rtol=0.0, atol=1.0e-12):
        sampled_temperature[0] = sampled_initial.temperature_c[0]
        sampled_moisture[0] = sampled_initial.moisture_concentration[0]
        next_output_index = 1

    final_time = float(output_times[-1])
    while current_time < final_time:
        chunk_end = min(current_time + chunk_duration_s, final_time)
        chunk_stop = int(np.searchsorted(output_times, chunk_end, side="right"))
        requested_times = output_times[next_output_index:chunk_stop]
        if requested_times.size and requested_times[0] <= current_time:
            raise RuntimeError("Chunked output indexing did not advance")
        append_chunk_end = (

```

```

        requested_times.size == 0
        or not np.isclose(requested_times[-1], chunk_end, rtol=0.0, atol=1.0e-12)
    )
    integration_times = (
        np.append(requested_times, chunk_end)
        if append_chunk_end
        else requested_times
    )
    chunk_solution = solve_problem2(
        history,
        radial_intervals=radial_intervals,
        output_times_s=integration_times,
        parameters=parameters,
        relative_tolerance=relative_tolerance,
        max_step_s=max_step_s,
        initial_time_s=None if current_temperature is None else current_time,
        initial_temperature_c=current_temperature,
        initial_moisture_concentration=current_moisture,
    )
    sampled_chunk = sample_solution(chunk_solution, requested_radius_cm)
    retained_count = requested_times.size
    if retained_count:
        sampled_temperature[next_output_index:chunk_stop] = (
            sampled_chunk.temperature_c[:retained_count]
        )
        sampled_moisture[next_output_index:chunk_stop] = (
            sampled_chunk.moisture_concentration[:retained_count]
        )
    current_temperature = chunk_solution.temperature_c[-1].copy()
    current_moisture = chunk_solution.moisture_concentration[-1].copy()
    current_time = chunk_end
    next_output_index = chunk_stop

if next_output_index != output_times.size:
    raise RuntimeError("Chunked solver did not populate every requested output time")
return Problem1Solution(
    time_s=output_times.copy(),
    radius_m=sampled_initial.radius_m,
    temperature_c=sampled_temperature,
    moisture_concentration=sampled_moisture,
)

```

problem3.py

```

"""Fixed-radius drying endpoint model for Problem 3."""

from __future__ import annotations

from dataclasses import dataclass

import numpy as np
from scipy.integrate import cumulative_simpson

from problem1 import (
    ChamberHistory,
    Problem1Parameters,
    Problem1Solution,
    nodal_control_volumes,
)
from problem2 import solve_problem2

@dataclass(frozen=True)
class Problem3Result:
    """Problem 3 solution together with its unrounded drying endpoint."""

    solution: Problem1Solution
    drying_time_s: float

```

```

strict_completion_time_s: float | None
maximum_moisture: float
maximum_radius_m: float
plateau_temperature_c: float
plateau_moisture_concentration: float
threshold: float
output_interval_s: float

def extend_chamber_history_to_plateau(
    history: ChamberHistory,
    end_time_s: float,
    *,
    averaging_window_s: float = 3600.0,
    transition_s: float = 60.0,
    plateau_temperature_c: float | None = None,
    plateau_moisture_concentration: float | None = None,
) -> ChamberHistory:
    """Extend Attachment 1 with a short transition to a constant tail plateau."""
    if not np.isfinite(end_time_s) or end_time_s <= history.time_s[-1]:
        raise ValueError("Extension end time must be later than the observed history")
    if not np.isfinite(averaging_window_s) or averaging_window_s <= 0.0:
        raise ValueError("Averaging window must be positive")
    if not np.isfinite(transition_s) or transition_s <= 0.0:
        raise ValueError("Plateau transition must be positive")

    tail = history.time_s >= history.time_s[-1] - averaging_window_s
    if np.count_nonzero(tail) < 2:
        raise ValueError("Averaging window must contain at least two observations")
    temperature = (
        float(np.mean(history.temperature_c[tail]))
        if plateau_temperature_c is None
        else float(plateau_temperature_c)
    )
    moisture = (
        float(np.mean(history.moisture_concentration[tail]))
        if plateau_moisture_concentration is None
        else float(plateau_moisture_concentration)
    )
    if not np.isfinite(temperature):
        raise ValueError("Plateau temperature must be finite")
    if not np.isfinite(moisture) or moisture < 0.0:
        raise ValueError("Plateau moisture concentration must be finite and non-negative")

    transition_time = min(history.time_s[-1] + transition_s, end_time_s)
    appended_times = [transition_time]
    if end_time_s > transition_time:
        appended_times.append(end_time_s)
    return ChamberHistory(
        time_s=np.concatenate([history.time_s, np.asarray(appended_times)]),
        temperature_c=np.concatenate(
            [history.temperature_c, np.full(len(appended_times), temperature)]
        ),
        moisture_concentration=np.concatenate(
            [history.moisture_concentration, np.full(len(appended_times), moisture)]
        ),
    )

def solve_problem3(
    history: ChamberHistory,
    radial_intervals: int,
    horizon_s: float,
    *,
    output_interval_s: float = 60.0,
    threshold: float = 0.15,
    averaging_window_s: float = 3600.0,
    transition_s: float = 60.0,
    plateau_temperature_c: float | None = None,
    plateau_moisture_concentration: float | None = None,

```

```

parameters: Problem1Parameters = Problem1Parameters(),
relative_tolerance: float = 1.0e-8,
max_step_s: float = 60.0,
) -> Problem3Result:
    """Continue the Problem 2 model until the full field reaches the threshold."""
    if not np.isfinite(output_interval_s) or output_interval_s <= 0.0:
        raise ValueError("Output interval must be positive")
    extended = extend_chamber_history_to_plateau(
        history,
        horizon_s,
        averaging_window_s=averaging_window_s,
        transition_s=transition_s,
        plateau_temperature_c=plateau_temperature_c,
        plateau_moisture_concentration=plateau_moisture_concentration,
    )
    regular_times = np.arange(
        output_interval_s,
        horizon_s + 0.5 * output_interval_s,
        output_interval_s,
    )
    regular_times = regular_times[regular_times <= horizon_s]
    if regular_times.size == 0 or regular_times[-1] < horizon_s:
        output_times = np.append(regular_times, horizon_s)
    else:
        output_times = regular_times
    solution = solve_problem2(
        extended,
        radial_intervals,
        output_times,
        parameters=parameters,
        relative_tolerance=relative_tolerance,
        max_step_s=max_step_s,
        maximum_moisture_threshold=threshold,
    )
    terminal_moisture = solution.moisture_concentration[-1]
    maximum_index = int(np.argmax(terminal_moisture))
    return Problem3Result(
        solution=solution,
        drying_time_s=float(solution.time_s[-1]),
        strict_completion_time_s=None,
        maximum_moisture=float(terminal_moisture[maximum_index]),
        maximum_radius_m=float(solution.radius_m[maximum_index]),
        plateau_temperature_c=float(extended.temperature_c[-1]),
        plateau_moisture_concentration=float(extended.moisture_concentration[-1]),
        threshold=float(threshold),
        output_interval_s=float(output_interval_s),
    )

def richardson_extrapolate_drying_time(
    coarse_time_s: float,
    fine_time_s: float,
    *,
    order: int = 2,
) -> float:
    """Extrapolate drying times from meshes whose spacing ratio is two."""
    if order <= 0:
        raise ValueError("Richardson order must be positive")
    if not np.isfinite(coarse_time_s) or not np.isfinite(fine_time_s):
        raise ValueError("Drying times must be finite")
    return float(fine_time_s + (fine_time_s - coarse_time_s) / (2.0*order - 1.0))

def validate_radially_nonincreasing_moisture(
    solution: Problem1Solution,
    *,
    tolerance: float = 1.0e-10,
) -> float:
    """Require every stored profile to be wettest at the centre and nonincreasing."""
    if not np.isfinite(tolerance) or tolerance < 0.0:

```

```

        raise ValueError("Radial monotonicity tolerance must be finite and non-negative")
    if solution.radius_m.size < 2:
        raise ValueError("Radial monotonicity requires at least two positions")
    maximum_outward_increase = float(
        np.max(np.diff(solution.moisture_concentration, axis=1))
    )
    if maximum_outward_increase > tolerance:
        raise ValueError(
            "Moisture profile contains an outward increase; centre-wettest claim failed"
        )
    return maximum_outward_increase

def moisture_balance_diagnostics(
    solution: Problem1Solution,
    history: ChamberHistory,
    *,
    parameters: Problem1Parameters = Problem1Parameters(),
) -> dict[str, float]:
    """Compare total moisture change with integrated convective surface flux."""
    if not np.isclose(solution.time_s[0], history.time_s[0], atol=1.0e-12, rtol=0.0):
        raise ValueError("Moisture balance requires the initial-time solution row")
    if solution.time_s[-1] > history.time_s[-1]:
        raise ValueError("Moisture balance times must lie within the chamber history")
    if not np.isclose(solution.radius_m[-1], parameters.radius_m, atol=1.0e-12, rtol=0.0):
        raise ValueError("Solution radius and model radius must match")

    volumes = nodal_control_volumes(solution.radius_m.size, parameters.radius_m)
    mean_moisture = solution.moisture_concentration @ volumes / np.sum(volumes)
    ambient_moisture = np.interp(
        solution.time_s,
        history.time_s,
        history.moisture_concentration,
    )
    mean_rate_from_surface = (
        -2.0
        * parameters.mass_transfer_coefficient_m_s
        / parameters.radius_m
        * (solution.moisture_concentration[:, -1] - ambient_moisture)
    )
    cumulative_surface_change = cumulative_simpson(
        mean_rate_from_surface,
        x=solution.time_s,
        initial=0.0,
    )
    residual = (mean_moisture - mean_moisture[0]) - cumulative_surface_change
    total_change = abs(float(mean_moisture[-1] - mean_moisture[0]))
    maximum_residual = float(np.max(np.abs(residual)))
    return {
        "maximum_absolute_residual": maximum_residual,
        "final_residual": float(residual[-1]),
        "relative_max_residual": maximum_residual / max(total_change, 1.0e-15),
        "total_mean_moisture_change": float(
            mean_moisture[-1] - mean_moisture[0]
        ),
        "integrated_surface_change": float(cumulative_surface_change[-1]),
    }

def truncate_solution_at_threshold(
    solution: Problem1Solution,
    *,
    threshold: float,
    output_interval_s: float,
) -> Problem1Solution:
    """Insert the threshold crossing and retain the first later strict regular row."""
    maximum = np.max(solution.moisture_concentration, axis=1)
    crossing_indices = np.flatnonzero(maximum <= threshold)
    if crossing_indices.size == 0 or crossing_indices[0] == 0:
        raise ValueError("Solution must bracket the first moisture threshold crossing")

```

```

upper = int(crossing_indices[0])
lower = upper - 1
# Interpolate each nodal field, then take the last nodal crossing. Merely
# interpolating the two maxima is wrong when the wettest node changes.
above = solution.moisture_concentration[lower] > threshold
nodal_drop = solution.moisture_concentration[lower, above] - solution.moisture_concentration[upper,
above]
fraction = float(np.max((solution.moisture_concentration[lower, above] - threshold) / nodal_drop))
crossing_time = solution.time_s[lower] + fraction * (
    solution.time_s[upper] - solution.time_s[lower]
)
crossing_temperature = solution.temperature_c[lower] + fraction * (
    solution.temperature_c[upper] - solution.temperature_c[lower]
)
crossing_moisture = solution.moisture_concentration[lower] + fraction * (
    solution.moisture_concentration[upper]
    - solution.moisture_concentration[lower]
)

on_regular_interval = np.isclose(
    np.mod(solution.time_s, output_interval_s),
    0.0,
    atol=1.0e-8,
)
regular_before = on_regular_interval & (
    solution.time_s < crossing_time - 1.0e-8
)
strict_candidates = np.flatnonzero(
    on_regular_interval
    & (solution.time_s > crossing_time + 1.0e-8)
    & (maximum < threshold)
)
if strict_candidates.size == 0:
    raise ValueError("Solution must include a later regular row strictly below threshold")
strict_index = int(strict_candidates[0])
return Problem1Solution(
    time_s=np.concatenate(
        [solution.time_s[regular_before], [crossing_time, solution.time_s[strict_index]]]
    ),
    radius_m=solution.radius_m.copy(),
    temperature_c=np.vstack(
        [
            solution.temperature_c[regular_before],
            crossing_temperature,
            solution.temperature_c[strict_index],
        ]
    ),
    moisture_concentration=np.vstack(
        [
            solution.moisture_concentration[regular_before],
            crossing_moisture,
            solution.moisture_concentration[strict_index],
        ]
    ),
)

def problem3_payload(result: Problem3Result) -> dict[str, object]:
    """Convert a sampled Problem 3 result to the result3.xlsx data schema."""
    solution = result.solution
    if solution.time_s.size == 0 or np.any(np.diff(solution.time_s) <= 0.0):
        raise ValueError("Problem 3 output times must be non-empty and increasing")
    if result.strict_completion_time_s is None:
        raise ValueError("Strict completion time requires a verified post-threshold row")
    if not np.isclose(
        solution.time_s[-1],
        result.strict_completion_time_s,
        rtol=0.0,
        atol=1.0e-8,
    ):

```

```

        raise ValueError("The final solution row must be the strict completion time")
critical_indices = np.flatnonzero(
    np.isclose(
        solution.time_s,
        result.drying_time_s,
        rtol=0.0,
        atol=1.0e-8,
    )
)
if critical_indices.size != 1:
    raise ValueError("The solution must contain exactly one critical threshold row")
critical_index = int(critical_indices[0])
regular_rows = np.ones(solution.time_s.size, dtype=bool)
regular_rows[critical_index] = False
if not np.allclose(
    np.mod(solution.time_s[regular_rows], result.output_interval_s),
    0.0,
    atol=1.0e-8,
):
    raise ValueError("All non-critical output rows must lie on the regular interval")
critical_maximum = float(
    np.max(solution.moisture_concentration[critical_index])
)
strict_maximum = float(np.max(solution.moisture_concentration[-1]))
if not np.isclose(critical_maximum, result.threshold, atol=1.0e-8, rtol=0.0):
    raise ValueError("Critical maximum moisture must equal the unrounded threshold")
if not strict_maximum < result.threshold:
    raise ValueError("Strict completion maximum moisture must be below threshold")

time_values: list[int | float] = []
for value in solution.time_s:
    rounded_integer = round(float(value))
    if np.isclose(value, rounded_integer, rtol=0.0, atol=1.0e-9):
        time_values.append(int(rounded_integer))
    else:
        time_values.append(round(float(value), 6))
return {
    "time_s": time_values,
    "radius_cm": np.round(solution.radius_m * 100.0, 10).tolist(),
    "moisture_concentration": np.round(
        solution.moisture_concentration,
        4,
    ).tolist(),
    "drying_time_s": round(result.drying_time_s, 6),
    "drying_time_h": round(result.drying_time_s / 3600.0, 10),
    "strict_completion_time_s": round(result.strict_completion_time_s, 6),
    "strict_completion_time_h": round(
        result.strict_completion_time_s / 3600.0,
        10,
    ),
    "threshold": result.threshold,
    "critical_maximum_moisture_unrounded": critical_maximum,
    "strict_completion_maximum_moisture_unrounded": strict_maximum,
    "maximum_radius_m": result.maximum_radius_m,
    "plateau_temperature_c": result.plateau_temperature_c,
    "plateau_moisture_concentration": result.plateau_moisture_concentration,
}

```

problem4.py

"""Moving-radius drying model for Problem 4.

The measured radius history is used as a prescribed moving boundary. The radial coordinate is mapped to $\xi=r/R(t)$; for dry-basis concentration the solid motion then cancels the grid velocity and leaves a diffusion equation on the fixed interval $0 \leq \xi \leq 1$ under spatially uniform dry-solid density. Appendix 4 density is used as an effective thermal-storage law; compatibility with a literal wet-bulk-density interpretation is audited separately.

```

"""

from __future__ import annotations

import csv
from dataclasses import dataclass, field
from pathlib import Path

import numpy as np
from numpy.typing import ArrayLike, NDArray
from scipy.integrate import cumulative_simpson, solve_ivp
from scipy.interpolate import PchipInterpolator
from scipy.sparse import bmat, diags

from problem1 import (
    ChamberHistory,
    Problem1Parameters,
    interpolate_history,
    nodal_control_volumes,
    radial_flux_divergence,
)

@dataclass(frozen=True)
class RadiusHistory:
    """Measured radius history in seconds and centimetres converted to metres."""

    time_s: NDArray[np.float64]
    radius_m: NDArray[np.float64]
    interpolation: str = "linear"
    _pchip: PchipInterpolator | None = field(init=False, repr=False, compare=False)

    def __post_init__(self) -> None:
        time = np.asarray(self.time_s, dtype=float)
        radius = np.asarray(self.radius_m, dtype=float)
        if time.ndim != 1 or radius.shape != time.shape or time.size < 2:
            raise ValueError("Radius history columns must be one-dimensional and equal-sized")
        if np.any(~np.isfinite(time)) or np.any(~np.isfinite(radius)):
            raise ValueError("Radius history must be finite")
        if np.any(np.diff(time) <= 0.0):
            raise ValueError("Radius times must be strictly increasing")
        if np.any(radius <= 0.0) or np.any(np.diff(radius) > 1.0e-12):
            raise ValueError("Radius must remain positive and non-increasing")
        if self.interpolation not in ("linear", "pchip"):
            raise ValueError("Radius interpolation must be linear or pchip")
        object.__setattr__(self, "time_s", time)
        object.__setattr__(self, "radius_m", radius)
        object.__setattr__(self, "_pchip", PchipInterpolator(time, radius) if self.interpolation ==
"pchip" else None)

@dataclass(frozen=True)
class Problem4Solution:
    """Full solution on the dimensionless moving-boundary coordinate."""

    time_s: NDArray[np.float64]
    xi: NDArray[np.float64]
    temperature_c: NDArray[np.float64]
    moisture_concentration: NDArray[np.float64]

def load_radius_history_csv(path: str | Path) -> RadiusHistory:
    """Load Attachment 2 exported as UTF-8 CSV."""
    rows: list[tuple[float, float]] = []
    with Path(path).open("r", encoding="utf-8-sig", newline="") as handle:
        reader = csv.DictReader(handle)
        if reader.fieldnames is None:
            raise ValueError("Attachment 2 has no header")
        names = set(reader.fieldnames)
        time_name = next((name for name in ("时间", "time", "time_s", "t") if name in names), None)

```

```

        radius_name = next((name for name in ("半径", "radius", "radius_cm", "R") if name in names),
None)
    # The official workbook may be exported with a locale-specific or
    # mojibake header. Attachment 2 has exactly two columns, so a
    # positional fallback is unambiguous and keeps the data auditable.
    if time_name is None or radius_name is None:
        time_name, radius_name = reader.fieldnames[:2]
    for line_number, row in enumerate(reader, start=2):
        try:
            rows.append((float(row[time_name]), float(row[radius_name])))
        except (TypeError, ValueError) as exc:
            raise ValueError(f"Invalid numeric value on CSV line {line_number}") from exc
    if not rows:
        raise ValueError("Attachment 2 contains no data rows")
    values = np.asarray(rows, dtype=float)
    return RadiusHistory(values[:, 0], values[:, 1] * 0.01)

def interpolate_radius(time_s: ArrayLike, history: RadiusHistory) -> NDArray[np.float64]:
    """Interpolate radius with the selected method and hold the measured tail."""
    query = np.asarray(time_s, dtype=float)
    if np.any(~np.isfinite(query)) or np.any(query < history.time_s[0]):
        raise ValueError("Requested radius time precedes Attachment 2")
    if history._pchip is not None:
        return history._pchip(np.minimum(query, history.time_s[-1]))
    return np.interp(query, history.time_s, history.radius_m)

def _concentration_array(concentration: ArrayLike) -> NDArray[np.float64]:
    values = np.asarray(concentration, dtype=float)
    if np.any(~np.isfinite(values)) or np.any(values < 0.0):
        raise ValueError("Moisture concentration must be finite and non-negative")
    return values

def density_q4(concentration: ArrayLike) -> NDArray[np.float64]:
    values = _concentration_array(concentration)
    return 760.0 + 90.0 * values

def heat_capacity_q4(concentration: ArrayLike) -> NDArray[np.float64]:
    values = _concentration_array(concentration)
    return 1850.0 + 2150.0 * values / (values + 1.0)

def thermal_conductivity_q4(concentration: ArrayLike) -> NDArray[np.float64]:
    values = _concentration_array(concentration)
    return 0.12 + 0.20 * values / (values + 1.0)

def diffusivity_q4(
    concentration: ArrayLike,
    temperature_c: ArrayLike,
) -> NDArray[np.float64]:
    moisture = _concentration_array(concentration)
    temperature_k = np.asarray(temperature_c, dtype=float) + 273.15
    if np.any(~np.isfinite(temperature_k)) or np.any(temperature_k <= 0.0):
        raise ValueError("Temperature must be finite and above absolute zero")
    if np.any(moisture <= 0.0):
        raise ValueError("Moisture concentration must be positive for diffusivity")
    moisture, temperature_k = np.broadcast_arrays(moisture, temperature_k)
    return 4.2e-4 * np.exp(-0.30 / moisture) * np.exp(-3850.0 / temperature_k)

def solve_problem4(
    history: ChamberHistory,
    radius_history: RadiusHistory,
    radial_intervals: int,
    output_times_s: ArrayLike,
    *,

```

```

parameters: Problem1Parameters = Problem1Parameters(),
relative_tolerance: float = 2.0e-8,
max_step_s: float = 60.0,
maximum_moisture_threshold: float | None = None,
initial_time_s: float | None = None,
initial_temperature_c: ArrayLike | None = None,
initial_moisture_concentration: ArrayLike | None = None,
) -> Problem4Solution:
    """Integrate the coupled Appendix 4 equations on ``xi=r/R(t)``."""
    if radial_intervals < 2:
        raise ValueError("At least two radial intervals are required")
    output_times = np.asarray(output_times_s, dtype=float)
    if output_times.ndim != 1 or output_times.size == 0 or np.any(~np.isfinite(output_times)):
        raise ValueError("Output times must be a non-empty finite array")
    if np.any(np.diff(output_times) <= 0.0):
        raise ValueError("Output times must be strictly increasing")
    start_time = history.time_s[0] if initial_time_s is None else float(initial_time_s)
    if not np.isfinite(start_time) or start_time < max(history.time_s[0], radius_history.time_s[0]) or
start_time > output_times[0]:
        raise ValueError("Initial time must not be after the first output time")
    if output_times[-1] > history.time_s[-1]:
        raise ValueError("Output times exceed the supplied chamber history")
    if maximum_moisture_threshold is not None and not (
        0.0 < maximum_moisture_threshold < parameters.initial_moisture_concentration
    ):
        raise ValueError("Maximum moisture threshold must lie between zero and the initial value")

    node_count = radial_intervals + 1
    xi = np.linspace(0.0, 1.0, node_count)
    tri = diags(
        [np.ones(node_count - 1), np.ones(node_count), np.ones(node_count - 1)],
        offsets=[-1, 0, 1], shape=(node_count, node_count), format="csr",
    )
    sparsity = bmat([[tri, tri], [tri, tri]], format="csr")

    supplied_initial = (initial_temperature_c is not None, initial_moisture_concentration is not None)
    if any(supplied_initial) and not all(supplied_initial):
        raise ValueError("Both initial fields must be supplied together")
    if all(supplied_initial):
        temperature_initial = np.asarray(initial_temperature_c, dtype=float)
        moisture_initial = np.asarray(initial_moisture_concentration, dtype=float)
        if temperature_initial.shape != (node_count,) or moisture_initial.shape != (node_count,):
            raise ValueError("Initial fields have the wrong nodal shape")
        if np.any(~np.isfinite(temperature_initial)) or np.any(~np.isfinite(moisture_initial)) or
np.any(moisture_initial <= 0.0):
            raise ValueError("Initial fields must be finite and moisture must be positive")
    else:
        temperature_initial = np.full(node_count, parameters.initial_temperature_c)
        moisture_initial = np.full(node_count, parameters.initial_moisture_concentration)

    def rhs(time_s: float, state: NDArray[np.float64]) -> NDArray[np.float64]:
        temperature = state[:node_count]
        concentration = np.exp(state[node_count:])
        radius = float(interpolate_radius(time_s, radius_history))
        density = density_q4(concentration)
        heat_capacity = heat_capacity_q4(concentration)
        conductivity = thermal_conductivity_q4(concentration)
        diffusivity = diffusivity_q4(concentration, temperature)
        ambient_temperature = interpolate_history(time_s, history.time_s, history.temperature_c)
        ambient_moisture = interpolate_history(time_s, history.time_s, history.moisture_concentration)
        temperature_div = radial_flux_divergence(
            temperature, conductivity, 1.0, radius * parameters.heat_transfer_coefficient_w_m2_k,
            ambient_temperature,
        ) / radius**2
        moisture_div = radial_flux_divergence(
            concentration, diffusivity, 1.0, radius * parameters.mass_transfer_coefficient_m_s,
            ambient_moisture,
        ) / radius**2
        return np.concatenate([temperature_div / (density * heat_capacity), moisture_div /
concentration])

```

```

initial_state = np.concatenate([temperature_initial, np.log(moisture_initial)])
atol = np.concatenate([np.full(node_count, 1.0e-8), np.full(node_count, 1.0e-10)])
event = None
if maximum_moisture_threshold is not None:
    def event(time_s: float, state: NDArray[np.float64]) -> float:
        del time_s
        return float(np.max(np.exp(state[node_count:])) - maximum_moisture_threshold)
    event.terminal = True
    event.direction = -1.0

result = solve_ivp(
    rhs, (start_time, float(output_times[-1])), initial_state, method="BDF", t_eval=output_times,
    rtol=relative_tolerance, atol=atol, max_step=max_step_s, jac_sparsity=sparsity, events=event,
)
if not result.success:
    raise RuntimeError(f"Problem 4 solver failed: {result.message}")
result_time = result.t
result_state = result.y
if maximum_moisture_threshold is not None:
    if not result.t_events or result.t_events[0].size == 0:
        raise RuntimeError("Maximum moisture threshold was not reached")
    event_time = float(result.t_events[0][0])
    event_state = result.y_events[0][0]
    if result_time.size and np.isclose(result_time[-1], event_time, atol=1.0e-9, rtol=0.0):
        result_time = result_time.copy(); result_state = result_state.copy()
        result_time[-1] = event_time; result_state[:, -1] = event_state
    else:
        result_time = np.append(result_time, event_time)
        result_state = np.column_stack([result_state, event_state])
return Problem4Solution(
    time_s=np.asarray(result_time), xi=xi,
    temperature_c=result_state[:node_count].T,
    moisture_concentration=np.exp(result_state[node_count:].T),
)

def sample_moving_solution(
    solution: Problem4Solution,
    radius_history: RadiusHistory,
    radius_cm: ArrayLike,
) -> NDArray[np.float64]:
    """Sample moisture at fixed physical radii; outside cells are NaN."""
    requested = np.asarray(radius_cm, dtype=float)
    if requested.ndim != 1 or requested.size == 0 or np.any(~np.isfinite(requested)) or
    np.any(requested < 0.0) or np.any(np.diff(requested) <= 0.0):
        raise ValueError("Requested radii must be strictly increasing")
    radii_m = requested * 0.01
    sampled = np.full((solution.time_s.size, requested.size), np.nan, dtype=float)
    current_radius = interpolate_radius(solution.time_s, radius_history)
    for row, radius in enumerate(current_radius):
        valid = radii_m <= radius + 1.0e-12
        if np.any(valid):
            coordinates = radii_m[valid] / radius
            sampled[row, valid] = np.interp(coordinates, solution.xi,
            solution.moisture_concentration[row])
    return sampled

def sample_surface(solution: Problem4Solution) -> NDArray[np.float64]:
    """Return the actual moving-surface concentration."""
    return solution.moisture_concentration[:, -1].copy()

def validate_radially_nonincreasing_moisture(
    solution: Problem4Solution,
    *,
    tolerance: float = 1.0e-10,
) -> float:
    """Require the stored moving-coordinate profiles to decrease outward."""

```

```

    if not np.isfinite(tolerance) or tolerance < 0.0:
        raise ValueError("Tolerance must be finite and non-negative")
    maximum_outward_increase = float(np.max(np.diff(solution.moisture_concentration, axis=1)))
    if maximum_outward_increase > tolerance:
        raise ValueError("Moisture profile contains an outward increase")
    return maximum_outward_increase

def moisture_balance_diagnostics(
    solution: Problem4Solution,
    chamber_history: ChamberHistory,
    radius_history: RadiusHistory,
    *,
    parameters: Problem1Parameters = Problem1Parameters(),
) -> dict[str, float]:
    """Check normalized-domain moisture change against the moving surface flux."""
    if not np.isclose(solution.time_s[0], chamber_history.time_s[0], atol=1.0e-12, rtol=0.0):
        raise ValueError("Moisture balance requires the initial-time row")
    volumes = nodal_control_volumes(solution.xi.size, 1.0)
    mean_moisture = solution.moisture_concentration @ volumes / np.sum(volumes)
    ambient = np.interp(
        solution.time_s,
        chamber_history.time_s,
        chamber_history.moisture_concentration,
    )
    current_radius = interpolate_radius(solution.time_s, radius_history)
    mean_rate_from_surface = (
        -2.0
        * parameters.mass_transfer_coefficient_m_s
        / current_radius
        * (solution.moisture_concentration[:, -1] - ambient)
    )
    integrated_change = cumulative_simpson(
        mean_rate_from_surface,
        x=solution.time_s,
        initial=0.0,
    )
    residual = (mean_moisture - mean_moisture[0]) - integrated_change
    total_change = abs(float(mean_moisture[-1] - mean_moisture[0]))
    maximum_residual = float(np.max(np.abs(residual)))
    return {
        "maximum_absolute_residual": maximum_residual,
        "final_residual": float(residual[-1]),
        "relative_max_residual": maximum_residual / max(total_change, 1.0e-15),
        "total_mean_moisture_change": float(mean_moisture[-1] - mean_moisture[0]),
        "integrated_surface_change": float(integrated_change[-1]),
    }

def density_shrinkage_compatibility(
    solution: Problem4Solution,
    radius_history: RadiusHistory,
) -> dict[str, object]:
    """Audit the hypothetical interpretation of Appendix 4 rho as wet density.

    This is separate from the moisture PDE flux balance. With fixed length,
    dry mass per unit length would be  $2\pi R^2 \int \rho(C)/(1+C) \xi d\xi$ .
    The first field must be the initial field; no conservation is imposed here.
    """
    if not np.isclose(solution.time_s[0], radius_history.time_s[0], atol=1.0e-12, rtol=0.0):
        raise ValueError("Density compatibility requires the initial-time row")
    volumes = nodal_control_volumes(solution.xi.size, 1.0)
    concentration = solution.moisture_concentration
    implied_dry_density = density_q4(concentration) / (1.0 + concentration)
    radii = interpolate_radius(solution.time_s, radius_history)
    # Unit-radius annular volumes already include pi (their sum is pi).
    mass_per_length = radii**2 * (implied_dry_density @ volumes)
    ratio = mass_per_length / mass_per_length[0]
    return {
        "interpretation": "hypothetical wet bulk density; constant cylinder length",
    }

```

```

        "time_s": solution.time_s.tolist(),
        "implied_dry_mass_per_length_kg_m": mass_per_length.tolist(),
        "mass_ratio_to_initial": ratio.tolist(),
        "final_mass_ratio": float(ratio[-1]),
        "maximum_absolute_ratio_deviation": float(np.max(np.abs(ratio - 1.0))),
    }

```

problem4_independent.py

"""Independent cell-centred finite-volume check of the Problem 4 model.

This implements the same continuum assumptions with different spatial unknowns and boundary closure. It does not call the production solver, its material-law functions, or its flux-divergence routine. Unknowns approximate point values at the midpoints of N complete annular cells; neither the axis nor the surface is a state node. Centre values are reconstructed by even quadratic extrapolation.

```

from __future__ import annotations

```

```

from dataclasses import dataclass

```

```

import numpy as np
from numpy.typing import ArrayLike, NDArray
from scipy.integrate import solve_ivp
from scipy.sparse import bmat, diags

```

```

from problem1 import ChamberHistory, Problem1Parameters
from problem4 import RadiusHistory

```

```

@dataclass(frozen=True)

```

```

class IndependentSolution:

```

```

    time_s: NDArray[np.float64]
    xi_centres: NDArray[np.float64]
    temperature_c: NDArray[np.float64]
    moisture: NDArray[np.float64]
    surface_temperature_c: NDArray[np.float64]
    surface_moisture: NDArray[np.float64]
    centre_moisture: NDArray[np.float64]
    drying_time_s: float | None
    endpoint_cell_maximum: float | None
    endpoint_reconstructed_maximum: float | None

```

```

def appendix4_coefficients(moisture: ArrayLike, temperature_c: ArrayLike):

```

```

    """Transcribe Appendix 4 separately from the production implementation."""
    concentration = np.asarray(moisture, dtype=float)
    kelvin = np.asarray(temperature_c, dtype=float) + 273.15
    if np.any(concentration <= 0.0) or np.any(kelvin <= 0.0):
        raise ValueError("Positive concentration and absolute temperature required")
    rho_cp = (760.0 + 90.0 * concentration) * (
        1850.0 + 2150.0 * concentration / (concentration + 1.0)
    )
    conductivity = 0.12 + 0.20 * concentration / (concentration + 1.0)
    diffusion = 4.2e-4 * np.exp(-0.30 / concentration - 3850.0 / kelvin)
    return rho_cp, conductivity, diffusion

```

```

def reconstruct_centre(values: ArrayLike) -> NDArray[np.float64]:

```

```

    """Even quadratic continuation from xi=dx/2 and 3dx/2 to xi=0.

```

```

    These are point-centred unknowns, not volume averages. For a smooth even
    exact profile this formula's reconstruction error is  $O(dx^4)$ ; the solved
    point values still carry the spatial scheme's discretization error.
    """

```

```

    field = np.asarray(values, dtype=float)
    if field.shape[-1] < 2:

```

```

        raise ValueError("At least two cells are needed for axis reconstruction")
    return (9.0 * field[..., 0] - field[..., 1]) / 8.0

def surface_from_series_resistance(
    last_value: ArrayLike,
    last_coefficient: ArrayLike,
    radius_m: ArrayLike,
    cell_count: int,
    exchange: float,
    ambient: ArrayLike,
) -> tuple[NDArray[np.float64], NDArray[np.float64]]:
    """Return actual-surface value and outward flux using a half-cell resistor.

     $J = (u_{\text{last}} - u_{\text{inf}}) / (R \cdot dx / (2 \cdot a_{\text{last}}) + 1/h)$ ,  $u_{\text{surface}} = u_{\text{inf}} + J/h$ .
    The coefficient is frozen at the last cell centre. That local closure is
    a truncation approximation, checked by spatial convergence, not an exact
    nonlinear surface solve.
    """
    last = np.asarray(last_value, dtype=float)
    coefficient = np.asarray(last_coefficient, dtype=float)
    ambient = np.asarray(ambient, dtype=float)
    if cell_count < 2 or exchange < 0.0 or np.any(coefficient <= 0.0):
        raise ValueError("Invalid boundary conductance or cell count")
    if exchange == 0.0:
        return last.copy(), np.zeros_like(last)
    resistance_ratio = exchange * np.asarray(radius_m) / (2.0 * cell_count * coefficient)
    surface = (last + resistance_ratio * ambient) / (1.0 + resistance_ratio)
    return surface, exchange * (surface - ambient)

def cell_flux_divergence(
    values: ArrayLike,
    coefficient: ArrayLike,
    radius_m: float,
    exchange: float,
    ambient: float,
) -> tuple[NDArray[np.float64], float]:
    """Annular flux balance in xi with arithmetic internal-face coefficients."""
    field = np.asarray(values, dtype=float)
    transport = np.asarray(coefficient, dtype=float)
    if field.ndim != 1 or field.shape != transport.shape or radius_m <= 0.0:
        raise ValueError("Mismatched fields or nonpositive radius")
    cell_count = field.size
    dx = 1.0 / cell_count
    faces = np.arange(cell_count + 1, dtype=float) * dx
    volumes = 0.5 * (faces[1:] ** 2 - faces[:-1] ** 2)
    flux = np.zeros(cell_count + 1)
    flux[1:-1] = (
        faces[1:-1]
        * 0.5 * (transport[:-1] + transport[1:])
        * np.diff(field) / dx
    )
    surface, outward = surface_from_series_resistance(
        field[-1], transport[-1], radius_m, cell_count, exchange, ambient
    )
    # flux stores  $\xi \cdot a \cdot u_{\xi}$ . At  $\xi=1$  this equals  $-R \cdot J$ .
    flux[-1] = -radius_m * float(outward)
    return np.diff(flux) / (radius_m ** 2 * volumes), float(surface)

def solve_independent_problem4(
    history: ChamberHistory,
    radius_history: RadiusHistory,
    cell_count: int,
    output_times_s: ArrayLike,
    *,
    threshold: float | None = 0.15,
    parameters: Problem1Parameters = Problem1Parameters(),
    rtol: float = 2.0e-9,

```

```

    max_step_s: float = 60.0,
) -> IndependentSolution:
    """Solve directly for T,C and terminate on the reconstructed global maximum.

    The checked set includes all cell-centred values, the even-reconstructed
    axis and the surface. In this monotone drying case its maximum is the
    reconstructed axis. This avoids mistaking  $\xi=dx/2$  for the actual centre.
    """
    times = np.asarray(output_times_s, dtype=float)
    if cell_count < 2 or times.ndim != 1 or times.size == 0:
        raise ValueError("At least two cells and nonempty output times required")
    if np.any(np.diff(times) <= 0.0) or times[0] < history.time_s[0]:
        raise ValueError("Invalid output-time ordering")
    if times[-1] > min(history.time_s[-1], radius_history.time_s[-1]):
        raise ValueError("Output horizon exceeds supplied history")
    if threshold is not None and not (0.0 < threshold < parameters.initial_moisture_concentration):
        raise ValueError("Invalid moisture threshold")
    n = cell_count
    xi = (np.arange(n, dtype=float) + 0.5) / n
    tri = diags([np.ones(n - 1), np.ones(n), np.ones(n - 1)], [-1, 0, 1], format="csr")
    sparsity = bmat([[tri, tri], [tri, tri]], format="csr")

    def surroundings(time_s):
        radius = float(np.interp(time_s, radius_history.time_s, radius_history.radius_m))
        ambient_t = float(np.interp(time_s, history.time_s, history.temperature_c))
        ambient_c = float(np.interp(time_s, history.time_s, history.moisture_concentration))
        return radius, ambient_t, ambient_c

    def rhs(time_s, state):
        temperature, moisture = state[:n], state[n:]
        capacity, conductivity, diffusivity = appendix4_coefficients(moisture, temperature)
        radius, ambient_t, ambient_c = surroundings(time_s)
        heat_div, _ = cell_flux_divergence(
            temperature, conductivity, radius,
            parameters.heat_transfer_coefficient_w_m2_k, ambient_t,
        )
        mass_div, _ = cell_flux_divergence(
            moisture, diffusivity, radius,
            parameters.mass_transfer_coefficient_m_s, ambient_c,
        )
        return np.concatenate([heat_div / capacity, mass_div])

    def reconstructed_maximum(time_s, state):
        temperature, moisture = state[:n], state[n:]
        _, _, diffusion = appendix4_coefficients(moisture[-1], temperature[-1])
        radius, _, ambient_c = surroundings(time_s)
        surface, _ = surface_from_series_resistance(
            moisture[-1], diffusion, radius, n,
            parameters.mass_transfer_coefficient_m_s, ambient_c,
        )
        return max(float(np.max(moisture)), float(reconstruct_centre(moisture)), float(surface))

    def event(time_s, state):
        return reconstructed_maximum(time_s, state) - threshold

    event.terminal = True
    event.direction = -1.0
    initial_state = np.concatenate([
        np.full(n, parameters.initial_temperature_c),
        np.full(n, parameters.initial_moisture_concentration),
    ])
    result = solve_ivp(
        rhs, (float(history.time_s[0]), float(times[-1])), initial_state,
        t_eval=times, method="BDF", rtol=rtol,
        atol=np.concatenate([np.full(n, 1.0e-9), np.full(n, 1.0e-11)]),
        max_step=max_step_s, jac_sparsity=sparsity,
        events=event if threshold is not None else None,
    )
    if not result.success:
        raise RuntimeError(result.message)

```

```

endpoint = cell_maximum = reconstructed_max = None
result_times, states = result.t, result.y
if threshold is not None:
    if not result.t_events[0].size:
        raise RuntimeError("Threshold not reached within supplied horizon")
    endpoint = float(result.t_events[0][0])
    endpoint_state = result.y_events[0][0]
    cell_maximum = float(np.max(endpoint_state[n:]))
    reconstructed_max = reconstructed_maximum(endpoint, endpoint_state)
    result_times = np.append(result_times, endpoint)
    states = np.column_stack([states, endpoint_state])
temperatures, moistures = states[:n].T, states[n:].T
radii = np.interp(result_times, radius_history.time_s, radius_history.radius_m)
ambient_ts = np.interp(result_times, history.time_s, history.temperature_c)
ambient_cs = np.interp(result_times, history.time_s, history.moisture_concentration)
_, last_k, last_d = appendix4_coefficients(moistures[:, -1], temperatures[:, -1])
surface_t, _ = surface_from_series_resistance(
    temperatures[:, -1], last_k, radii, n,
    parameters.heat_transfer_coefficient_w_m2_k, ambient_ts,
)
surface_c, _ = surface_from_series_resistance(
    moistures[:, -1], last_d, radii, n,
    parameters.mass_transfer_coefficient_m_s, ambient_cs,
)
return IndependentSolution(
    result_times, xi, temperatures, moistures, surface_t, surface_c,
    reconstruct_centre(moistures), endpoint, cell_maximum, reconstructed_max,
)

def sample_independent_moisture(
    solution: IndependentSolution,
    radius_history: RadiusHistory,
    physical_radii_cm: ArrayLike,
) -> NDArray[np.float64]:
    """Piecewise-linear interpolation including separately reconstructed ends."""
    radii_m = np.asarray(physical_radii_cm, dtype=float) * 0.01
    if radii_m.ndim != 1 or np.any(radii_m < 0.0):
        raise ValueError("Nonnegative one-dimensional physical radii required")
    result = np.full((solution.time_s.size, radii_m.size), np.nan)
    xi = np.concatenate([[0.0], solution.xi_centres, [1.0]])
    for row, time_s in enumerate(solution.time_s):
        radius = float(np.interp(time_s, radius_history.time_s, radius_history.radius_m))
        field = np.concatenate([
            [solution.centre_moisture[row]], solution.moisture[row],
            [solution.surface_moisture[row]],
        ])
        valid = radii_m <= radius + 1.0e-12
        result[row, valid] = np.interp(radii_m[valid] / radius, xi, field)
    return result

```

restore_full_result2.py

```

"""Restore the complete, rounded, per-second result2.xlsx from the lossless NPZ."""
from pathlib import Path
import argparse
import numpy as np
from openpyxl import Workbook
from openpyxl.cell import WriteOnlyCell

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--input', type=Path, default=Path('result2_full_process.npz'))
    parser.add_argument('--output', type=Path, default=Path('result2_full_process.xlsx'))
    args = parser.parse_args()
    if args.output.exists():
        raise FileExistsError(f'Refusing to overwrite {args.output}; choose another --output')
    wb = Workbook(write_only=True)

```

```

with np.load(args.input,allow_pickle=False) as archive:
    for name in ('温度','水分浓度'):
        ws = wb.create_sheet(name)
        ws.append(['时间\到药材中心的距离']+[i/10 for i in range(21)])
        for row in archive[name]:
            cells = [int(row[0])]
            for value in row[1:]:
                cell = WriteOnlyCell(ws,float(value)); cell.number_format='0.0000';
cells.append(cell)
        ws.append(cells)
args.output.parent.mkdir(parents=True,exist_ok=True)
wb.save(args.output)
print(args.output)

if __name__=='__main__': main()

```

run_problem1.py

```

"""Solve Problem 1 and export workbook-ready values plus diagnostic figures."""

from __future__ import annotations

import argparse
import json
from pathlib import Path

import matplotlib
import numpy as np

matplotlib.use("Agg")
import matplotlib.pyplot as plt

from problem1 import (
    Problem1Parameters,
    load_chamber_history_csv,
    richardson_extrapolate_solutions,
    result_payload,
    sample_solution,
    solve_problem1,
)

def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--input", type=Path, default=Path("attachment1.csv"))
    parser.add_argument("--output", type=Path, default=Path("problem1_result.json"))
    parser.add_argument("--figure-dir", type=Path, default=Path("."))
    parser.add_argument("--coarse-radial-intervals", type=int, default=5120)
    parser.add_argument("--fine-radial-intervals", type=int, default=10240)
    return parser

def save_figures(payload: dict[str, list], figure_dir: Path) -> None:
    figure_dir.mkdir(parents=True, exist_ok=True)
    time = np.asarray(payload["time_s"], dtype=float)
    radius = np.asarray(payload["radius_cm"], dtype=float)
    temperature = np.asarray(payload["temperature_c"], dtype=float)
    moisture = np.asarray(payload["moisture_concentration"], dtype=float)

    profile_times = [100, 300, 600, 900, 1200, 1500, 1800]
    colors = plt.cm.viridis(np.linspace(0.05, 0.95, len(profile_times)))
    fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2), constrained_layout=True)
    for selected_time, color in zip(profile_times, colors, strict=True):
        row = selected_time - 1
        axes[0].plot(radius, temperature[row], color=color, label=f"{selected_time} s")
        axes[1].plot(radius, moisture[row], color=color, label=f"{selected_time} s")
    axes[0].set(xlabel="Radius r (cm)", ylabel="Temperature (deg C)")
    axes[1].set(xlabel="Radius r (cm)", ylabel="Moisture concentration (kg/kg)")

```

```

for axis in axes:
    axis.grid(alpha=0.25)
axes[1].legend(ncol=2, fontsize=8)
fig.savefig(figure_dir / "problem1_radial_profiles.png", dpi=220)
plt.close(fig)

locations = [(0, "center"), (10, "mid-radius"), (20, "surface")]
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2), constrained_layout=True)
for column, label in locations:
    axes[0].plot(time, temperature[:, column], label=label)
    axes[1].plot(time, moisture[:, column], label=label)
axes[0].set(xlabel="Time (s)", ylabel="Temperature (deg C)")
axes[1].set(xlabel="Time (s)", ylabel="Moisture concentration (kg/kg)")
for axis in axes:
    axis.grid(alpha=0.25)
    axis.legend()
fig.savefig(figure_dir / "problem1_time_histories.png", dpi=220)
plt.close(fig)

def main() -> None:
    args = build_parser().parse_args()
    if args.fine_radial_intervals != 2 * args.coarse_radial_intervals:
        raise ValueError("Fine radial grid must have twice as many intervals as the coarse grid")
    history = load_chamber_history_csv(args.input)
    output_times = np.arange(1.0, 1801.0)
    requested_radii_cm = np.arange(0.0, 2.0 + 0.05, 0.1)
    sampled_solutions = []
    for radial_intervals in (
        args.coarse_radial_intervals,
        args.fine_radial_intervals,
    ):
        solution = solve_problem1(
            history,
            radial_intervals,
            output_times,
            parameters=Problem1Parameters(),
            relative_tolerance=5.0e-10,
            max_step_s=1.0,
        )
        sampled_solutions.append(sample_solution(solution, requested_radii_cm))
    extrapolated = richardson_extrapolate_solutions(
        sampled_solutions[0],
        sampled_solutions[1],
        order=2,
    )
    payload = result_payload(extrapolated)
    args.output.parent.mkdir(parents=True, exist_ok=True)
    args.output.write_text(
        json.dumps(payload, ensure_ascii=False, separators=(",", ":")),
        encoding="utf-8",
    )
    save_figures(payload, args.figure_dir)

if __name__ == "__main__":
    main()

```

run_problem2.py

"""Solve Problem 2 and export workbook-ready values plus diagnostic figures."""

```

from __future__ import annotations

import argparse
import json
from pathlib import Path

```

```

import matplotlib
import numpy as np

matplotlib.use("Agg")
import matplotlib.pyplot as plt

from problem1 import (
    load_chamber_history_csv,
    result_payload,
    richardson_extrapolate_solutions,
)
from problem2 import diffusivity_q2, solve_problem2_sampled_in_chunks

def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--input", type=Path, default=Path("attachment1.csv"))
    parser.add_argument("--output", type=Path, default=Path("problem2_result.json"))
    parser.add_argument("--figure-dir", type=Path, default=Path("."))
    parser.add_argument("--coarse-radial-intervals", type=int, default=2560)
    parser.add_argument("--fine-radial-intervals", type=int, default=5120)
    parser.add_argument("--chunk-duration-s", type=float, default=600.0)
    return parser

def save_figures(payload: dict[str, list], figure_dir: Path) -> None:
    figure_dir.mkdir(parents=True, exist_ok=True)
    time = np.asarray(payload["time_s"], dtype=float)
    radius = np.asarray(payload["radius_cm"], dtype=float)
    temperature = np.asarray(payload["temperature_c"], dtype=float)
    moisture = np.asarray(payload["moisture_concentration"], dtype=float)

    profile_times = [1800, 3600, 5400, 7200, 9000, 10800]
    colors = plt.cm.viridis(np.linspace(0.05, 0.95, len(profile_times)))
    fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2), constrained_layout=True)
    for selected_time, color in zip(profile_times, colors, strict=True):
        row = selected_time - 1
        label = f"{selected_time / 3600:g} h"
        axes[0].plot(radius, temperature[row], color=color, label=label)
        axes[1].plot(radius, moisture[row], color=color, label=label)
    axes[0].set(xlabel="Radius r (cm)", ylabel="Temperature (deg C)")
    axes[1].set(xlabel="Radius r (cm)", ylabel="Moisture concentration (kg/kg)")
    for axis in axes:
        axis.grid(alpha=0.25)
    axes[1].legend(ncol=2, fontsize=8)
    fig.savefig(figure_dir / "problem2_radial_profiles.png", dpi=220)
    plt.close(fig)

    locations = [(0, "center"), (10, "mid-radius"), (20, "surface")]
    fig, axes = plt.subplots(1, 3, figsize=(14.0, 4.2), constrained_layout=True)
    for column, label in locations:
        axes[0].plot(time / 3600.0, temperature[:, column], label=label)
        axes[1].plot(time / 3600.0, moisture[:, column], label=label)
        axes[2].plot(
            time / 3600.0,
            diffusivity_q2(moisture[:, column], temperature[:, column]),
            label=label,
        )
    axes[0].set(xlabel="Time (h)", ylabel="Temperature (deg C)")
    axes[1].set(xlabel="Time (h)", ylabel="Moisture concentration (kg/kg)")
    axes[2].set(xlabel="Time (h)", ylabel="Diffusivity (m2/s)")
    for axis in axes:
        axis.grid(alpha=0.25)
        axis.legend()
    fig.savefig(figure_dir / "problem2_time_histories.png", dpi=220)
    plt.close(fig)

def main() -> None:
    args = build_parser().parse_args()

```

```

if args.fine_radial_intervals != 2 * args.coarse_radial_intervals:
    raise ValueError("Fine radial grid must have twice as many intervals as the coarse grid")
history = load_chamber_history_csv(args.input)
output_times = np.arange(1.0, 10801.0)
sample_radius_cm = np.arange(0.0, 2.0 + 0.05, 0.1)
solutions = []
for radial_intervals in (
    args.coarse_radial_intervals,
    args.fine_radial_intervals,
):
    print(f"Solving Problem 2 on {radial_intervals} radial intervals...", flush=True)
    solutions.append(
        solve_problem2_sampled_in_chunks(
            history,
            radial_intervals=radial_intervals,
            output_times=output_times,
            sample_radius_cm=sample_radius_cm,
            chunk_duration_s=args.chunk_duration_s,
            relative_tolerance=2.0e-10,
            max_step_s=2.0,
        )
    )
    print(f"Completed {radial_intervals} radial intervals.", flush=True)
coarse, fine = solutions
extrapolated = richardson_extrapolate_solutions(coarse, fine, order=2)
payload = result_payload(extrapolated)
payload["numerical_method"] = {
    "coarse_radial_intervals": args.coarse_radial_intervals,
    "fine_radial_intervals": args.fine_radial_intervals,
    "richardson_order": 2,
    "relative_tolerance": 2.0e-10,
    "maximum_time_step_s": 2.0,
    "chunk_duration_s": args.chunk_duration_s,
    "max_abs_temperature_difference_c": float(
        np.max(np.abs(fine.temperature_c - coarse.temperature_c))
    ),
    "max_abs_moisture_difference": float(
        np.max(
            np.abs(
                fine.moisture_concentration
                - coarse.moisture_concentration
            )
        )
    ),
    "temperature_four_decimal_mismatches": int(
        np.count_nonzero(
            np.round(fine.temperature_c, 4)
            != np.round(coarse.temperature_c, 4)
        )
    ),
    "moisture_four_decimal_mismatches": int(
        np.count_nonzero(
            np.round(fine.moisture_concentration, 4)
            != np.round(coarse.moisture_concentration, 4)
        )
    ),
}
args.output.parent.mkdir(parents=True, exist_ok=True)
args.output.write_text(
    json.dumps(payload, ensure_ascii=False, separators=(",", ":")),
    encoding="utf-8",
)
save_figures(payload, args.figure_dir)

if __name__ == "__main__":
    main()

```

run_problem3.py

```
"""Solve Problem 3, locate the drying endpoint, and export workbook-ready values."""

from __future__ import annotations

import argparse
import gc
import json
from dataclasses import replace
from pathlib import Path

import matplotlib
import numpy as np

matplotlib.use("Agg")
import matplotlib.pyplot as plt

from problem1 import (
    load_chamber_history_csv,
    richardson_extrapolate_solutions,
    sample_solution,
)
from problem2 import solve_problem2
from problem3 import (
    Problem3Result,
    extend_chamber_history_to_plateau,
    moisture_balance_diagnostics,
    problem3_payload,
    richardson_extrapolate_drying_time,
    solve_problem3,
    truncate_solution_at_threshold,
    validate_radially_nonincreasing_moisture,
)

def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--input", type=Path, default=Path("attachment1.csv"))
    parser.add_argument("--output", type=Path, default=Path("problem3_result.json"))
    parser.add_argument("--figure-dir", type=Path, default=Path("."))
    parser.add_argument("--coarse-radial-intervals", type=int, default=2560)
    parser.add_argument("--fine-radial-intervals", type=int, default=5120)
    parser.add_argument("--sensitivity-radial-intervals", type=int, default=640)
    parser.add_argument("--horizon-hours", type=float, default=96.0)
    return parser

def save_figures(result: Problem3Result, figure_dir: Path) -> None:
    figure_dir.mkdir(parents=True, exist_ok=True)
    solution = result.solution
    time_h = solution.time_s / 3600.0
    radius_cm = solution.radius_m * 100.0
    moisture = solution.moisture_concentration

    selected_hours = [6.0, 12.0, 24.0, 36.0, 48.0]
    selected_hours = [hour for hour in selected_hours if hour < time_h[-1]]
    selected_indices = [int(np.argmax(np.abs(time_h - hour))) for hour in selected_hours]
    critical_index = int(
        np.flatnonzero(
            np.isclose(solution.time_s, result.drying_time_s, atol=1.0e-8, rtol=0.0)
        )[0]
    )
    selected_indices.extend([critical_index, len(time_h) - 1])
    selected_indices = list(dict.fromkeys(selected_indices))
    fig, axis = plt.subplots(figsize=(7.2, 4.6), constrained_layout=True)
    for index in selected_indices:
        if index == critical_index:
            label = "critical threshold"
```

```

        elif index == len(time_h) - 1:
            label = "first strict regular time"
        else:
            label = f"{time_h[index]:g} h"
            axis.plot(radius_cm, moisture[index], label=label)
axis.axhline(result.threshold, color="#B91C1C", linestyle="--", linewidth=1.0)
axis.set(
    xlabel="Radius r (cm)",
    ylabel="Moisture concentration (kg/kg)",
)
axis.grid(alpha=0.25)
axis.legend(ncol=2, fontsize=8)
fig.savefig(figure_dir / "problem3_radial_profiles.png", dpi=220)
plt.close(fig)

fig, axis = plt.subplots(figsize=(7.2, 4.6), constrained_layout=True)
for column, label in ((0, "center"), (10, "mid-radius"), (20, "surface")):
    axis.plot(time_h, moisture[:, column], label=label)
axis.axhline(result.threshold, color="#B91C1C", linestyle="--", linewidth=1.0)
axis.set(
    xlabel="Time (h)",
    ylabel="Moisture concentration (kg/kg)",
)
axis.grid(alpha=0.25)
axis.legend()
fig.savefig(figure_dir / "problem3_time_histories.png", dpi=220)
plt.close(fig)

def main() -> None:
    args = build_parser().parse_args()
    if args.fine_radial_intervals != 2 * args.coarse_radial_intervals:
        raise ValueError("Fine radial grid must have twice as many intervals as the coarse grid")
    history = load_chamber_history_csv(args.input)
    horizon_s = args.horizon_hours * 3600.0
    solver_options = {"relative_tolerance": 2.0e-9, "max_step_s": 60.0}

    endpoint_results = []
    for radial_intervals in (
        args.coarse_radial_intervals,
        args.fine_radial_intervals,
    ):
        endpoint = solve_problem3(
            history,
            radial_intervals,
            horizon_s,
            output_interval_s=3600.0,
            **solver_options,
        )
        endpoint_results.append(endpoint)
        print(
            f"endpoint grid={radial_intervals}: ",
            f"{endpoint.drying_time_s:.6f} s",
            flush=True,
        )
    extrapolated_time = richardson_extrapolate_drying_time(
        endpoint_results[0].drying_time_s,
        endpoint_results[1].drying_time_s,
        order=2,
    )

    common_end_time = extrapolated_time + 60.0
    extended_history = extend_chamber_history_to_plateau(history, common_end_time)
    regular_times = np.arange(60.0, common_end_time + 1.0e-9, 60.0)
    common_times = np.unique(
        np.append(
            regular_times,
            [extrapolated_time - 60.0, extrapolated_time + 60.0],
        )
    )

```

```

diagnostic_times = np.unique(
    np.concatenate([np.arange(0.0, 601.0), common_times])
)
requested_radial_cm = np.arange(0.0, 2.0 + 0.05, 0.1)
sampled_solutions = []
moisture_balance: dict[str, dict[str, float]] = {}
radial_monotonicity: dict[str, float] = {}
for radial_intervals in (
    args.coarse_radial_intervals,
    args.fine_radial_intervals,
):
    full_solution = solve_problem2(
        extended_history,
        radial_intervals,
        diagnostic_times,
        **solver_options,
    )
    moisture_balance[str(radial_intervals)] = moisture_balance_diagnostics(
        full_solution,
        extended_history,
    )
    radial_monotonicity[str(radial_intervals)] = (
        validate_radially_nonincreasing_moisture(full_solution)
    )
    sampled_full_solution = sample_solution(
        full_solution,
        requested_radial_cm,
    )
    common_indices = np.searchsorted(diagnostic_times, common_times)
    sampled_solutions.append(
        replace(
            sampled_full_solution,
            time_s=sampled_full_solution.time_s[common_indices],
            temperature_c=sampled_full_solution.temperature_c[common_indices],
            moisture_concentration=(
                sampled_full_solution.moisture_concentration[common_indices]
            ),
        )
    )
    del full_solution
    del sampled_full_solution
    gc.collect()
    print(f"sampld field grid={radial_intervals}", flush=True)
extrapolated_solution = richardson_extrapolate_solutions(
    sampled_solutions[0],
    sampled_solutions[1],
    order=2,
)
maximum_outward_increase = validate_radially_nonincreasing_moisture(
    extrapolated_solution
)
output_solution = truncate_solution_at_threshold(
    extrapolated_solution,
    threshold=0.15,
    output_interval_s=60.0,
)
critical_index = output_solution.time_s.size - 2
critical_moisture = output_solution.moisture_concentration[critical_index]
maximum_index = int(np.argmax(critical_moisture))
result = Problem3Result(
    solution=output_solution,
    drying_time_s=float(output_solution.time_s[critical_index]),
    strict_completion_time_s=float(output_solution.time_s[-1]),
    maximum_moisture=float(critical_moisture[maximum_index]),
    maximum_radius_m=float(output_solution.radius_m[maximum_index]),
    plateau_temperature_c=float(extended_history.temperature_c[-1]),
    plateau_moisture_concentration=float(
        extended_history.moisture_concentration[-1]
    ),
    threshold=0.15,

```

```

        output_interval_s=60.0,
    )

    tail_half_hour = history.time_s >= history.time_s[-1] - 1800.0
    tail_hour = history.time_s >= history.time_s[-1] - 3600.0
    temperature_mean = float(np.mean(history.temperature_c[tail_hour]))
    moisture_mean = float(np.mean(history.moisture_concentration[tail_hour]))
    temperature_std = float(np.std(history.temperature_c[tail_hour], ddof=1))
    moisture_std = float(np.std(history.moisture_concentration[tail_hour], ddof=1))
    scenarios = {
        "last_hour_mean": (temperature_mean, moisture_mean),
        "last_half_hour_mean": (
            float(np.mean(history.temperature_c[tail_half_hour])),
            float(np.mean(history.moisture_concentration[tail_half_hour])),
        ),
        "last_observation": (
            float(history.temperature_c[-1]),
            float(history.moisture_concentration[-1]),
        ),
        "cool_wet_one_standard_deviation": (
            temperature_mean - temperature_std,
            moisture_mean + moisture_std,
        ),
        "warm_dry_one_standard_deviation": (
            temperature_mean + temperature_std,
            max(0.0, moisture_mean - moisture_std),
        ),
    }
    sensitivity: dict[str, dict[str, float]] = {}
    for name, (plateau_temperature, plateau_moisture) in scenarios.items():
        scenario = solve_problem3(
            history,
            args.sensitivity_radial_intervals,
            horizon_s,
            output_interval_s=3600.0,
            plateau_temperature_c=plateau_temperature,
            plateau_moisture_concentration=plateau_moisture,
            **solver_options,
        )
        sensitivity[name] = {
            "plateau_temperature_c": plateau_temperature,
            "plateau_moisture_concentration": plateau_moisture,
            "drying_time_s": scenario.drying_time_s,
            "drying_time_h": scenario.drying_time_s / 3600.0,
        }
        print(
            f"sensitivity {name}: {scenario.drying_time_s / 3600.0:.6f} h",
            flush=True,
        )

    payload = problem3_payload(result)
    payload["grid_endpoint_s"] = {
        str(args.coarse_radial_intervals): endpoint_results[0].drying_time_s,
        str(args.fine_radial_intervals): endpoint_results[1].drying_time_s,
        "richardson": result.drying_time_s,
    }
    payload["moisture_balance"] = moisture_balance
    payload["radial_monotonicity"] = {
        "center_is_wettest_at_all_output_times": True,
        "maximum_outward_increase_by_grid": radial_monotonicity,
        "extrapolated_sampled_maximum_outward_increase": maximum_outward_increase,
    }
    payload["sensitivity"] = sensitivity
    args.output.parent.mkdir(parents=True, exist_ok=True)
    args.output.write_text(
        json.dumps(payload, ensure_ascii=False, separators=(",", ":"),
            encoding="utf-8",
        )
    )
    save_figures(result, args.figure_dir)

```

```
if __name__ == "__main__":
    main()
```

run_problem4.py

```
"""Solve Problem 4, perform grid checks, and create workbook-ready payloads."""
```

```
from __future__ import annotations
```

```
import argparse
import hashlib
import json
from pathlib import Path
```

```
import matplotlib
import numpy as np
```

```
matplotlib.use("Agg")
import matplotlib.pyplot as plt
```

```
from problem1 import load_chamber_history_csv
from problem3 import extend_chamber_history_to_plateau
from problem4 import (
    RadiusHistory,
    load_radius_history_csv,
    moisture_balance_diagnostics,
    sample_moving_solution,
    sample_surface,
    solve_problem4,
    validate_radially_nonincreasing_moisture,
)
```

```
def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--chamber", type=Path, default=Path("attachment1.csv"))
    parser.add_argument("--radius", type=Path, default=Path("attachment2.csv"))
    parser.add_argument("--output", type=Path, default=Path("problem4_result.json"))
    parser.add_argument("--figure-dir", type=Path, default=Path("."))
    parser.add_argument("--diagnostics-output", type=Path,
default=Path("problem4_production_diagnostics.json"))
    parser.add_argument("--coarse-radial-intervals", type=int, default=640)
    parser.add_argument("--fine-radial-intervals", type=int, default=1280)
    parser.add_argument("--horizon-hours", type=float, default=72.0)
    return parser
```

```
def richardson(coarse: np.ndarray, fine: np.ndarray, order: int = 2) -> np.ndarray:
    """Second-order Richardson extrapolation for equal output grids."""
    if coarse.shape != fine.shape:
        raise ValueError("Richardson inputs must have equal shapes")
    return fine + (fine - coarse) / (2.0*order - 1.0)
```

```
def interpolate_rows(values: np.ndarray, times: np.ndarray, query: float) -> np.ndarray:
    """Linearly interpolate each spatial column at one time."""
    if query < times[0] or query > times[-1]:
        raise ValueError("Interpolation query lies outside the solution")
    index = int(np.searchsorted(times, query))
    if index == 0:
        return values[0].copy()
    if index == times.size:
        return values[-1].copy()
    if np.isclose(times[index], query, atol=1.0e-10, rtol=0.0):
        return values[index].copy()
    fraction = (query - times[index - 1]) / (times[index] - times[index - 1])
    return values[index - 1] + fraction * (values[index] - values[index - 1])
```

```

def write_diagnostics(payload: dict, path: Path, chamber_path: Path, radius_path: Path) -> None:
    """Retain compact production evidence alongside the generated workbook."""
    omitted = {"time_s", "radius_cm", "moisture_concentration", "surface_moisture_concentration"}
    evidence = {key: value for key, value in payload.items() if key not in omitted}
    evidence["input_sha256"] = {p.as_posix(): hashlib.sha256(p.read_bytes()).hexdigest() for p in
(chamber_path, radius_path)}
    evidence["data_rows"] = len(payload["time_s"])
    evidence["table6_columns"] = ["time_h", "center", "0.5_cm", "1.0_cm", "surface"]
    evidence["table6"] = []
    for hour in range(6, 49, 6):
        if hour * 3600 not in payload["time_s"]:
            continue
        index = payload["time_s"].index(hour * 3600)
        row = payload["moisture_concentration"][index]
        evidence["table6"].append([hour, row[0], row[5], row[10],
payload["surface_moisture_concentration"][index]])
        path.parent.mkdir(parents=True, exist_ok=True)
        path.write_text(json.dumps(evidence, indent=2, ensure_ascii=False, allow_nan=False) + "\n",
encoding="utf-8")

def save_figures(
    times_s: np.ndarray,
    radius_cm: np.ndarray,
    moisture: np.ndarray,
    surface: np.ndarray,
    radius_history,
    drying_time_s: float,
    figure_dir: Path,
) -> None:
    figure_dir.mkdir(parents=True, exist_ok=True)
    time_h = times_s / 3600.0
    critical_index = int(np.argmin(np.abs(times_s - drying_time_s)))
    selected_hours = [6.0, 12.0, 24.0, 36.0, 48.0]
    selected = [int(np.argmin(np.abs(time_h - value))) for value in selected_hours if value <= time_h[-
1]]
    selected.extend([critical_index, len(times_s) - 1])
    selected = list(dict.fromkeys(selected))
    fig, axis = plt.subplots(figsize=(7.4, 4.7), constrained_layout=True)
    for index in selected:
        label = f"{time_h[index]:g} h"
        if np.isclose(times_s[index], drying_time_s, atol=1.0e-6, rtol=0.0):
            label = "critical threshold"
        elif index == len(times_s) - 1:
            label = "first strict regular time"
        axis.plot(radius_cm, moisture[index], marker="", label=label)
    axis.axhline(0.15, color="#B91C1C", linestyle="--", linewidth=1.0)
    axis.set(xlabel="Fixed physical radius (cm)", ylabel="Moisture concentration (kg/kg)")
    axis.grid(alpha=0.25)
    axis.legend(ncol=2, fontsize=8)
    fig.savefig(figure_dir / "problem4_radial_profiles.png", dpi=220)
    plt.close(fig)

    fig, axis = plt.subplots(figsize=(7.4, 4.7), constrained_layout=True)
    center = moisture[:, 0]
    axis.plot(time_h, center, label="center")
    axis.plot(time_h, surface, label="actual surface")
    axis.axhline(0.15, color="#B91C1C", linestyle="--", linewidth=1.0)
    axis.set(xlabel="Time (h)", ylabel="Moisture concentration (kg/kg)")
    axis.grid(alpha=0.25)
    axis.legend()
    fig.savefig(figure_dir / "problem4_time_histories.png", dpi=220)
    plt.close(fig)

    fig, axis = plt.subplots(figsize=(7.4, 4.7), constrained_layout=True)
    axis.plot(radius_history.time_s / 3600.0, radius_history.radius_m * 100.0, color="#1D4ED8")
    axis.set(xlabel="Time (h)", ylabel="Measured radius (cm)")
    axis.grid(alpha=0.25)

```

```

fig.savefig(figure_dir / "problem4_radius_history.png", dpi=220)
plt.close(fig)

def main() -> None:
    args = build_parser().parse_args()
    chamber = load_chamber_history_csv(args.chamber)
    radius_history = load_radius_history_csv(args.radius)
    if args.fine_radial_intervals != 2 * args.coarse_radial_intervals:
        raise ValueError("Second-order Richardson requires fine intervals = 2 * coarse intervals")
    horizon_s = float(args.horizon_hours * 3600.0)
    extended_chamber = extend_chamber_history_to_plateau(chamber, horizon_s)
    threshold = 0.15
    solver_options = {"relative_tolerance": 2.0e-8, "max_step_s": 120.0}
    endpoint_times: dict[str, float] = {}
    for grid in (args.coarse_radial_intervals, args.fine_radial_intervals):
        probe_times = np.unique(np.append(np.arange(3600.0, horizon_s, 3600.0), horizon_s))
        endpoint = solve_problem4(
            extended_chamber,
            radius_history,
            grid,
            probe_times,
            maximum_moisture_threshold=threshold,
            **solver_options,
        )
        endpoint_times[str(grid)] = float(endpoint.time_s[-1])
        print(f"endpoint grid={grid}: {endpoint.time_s[-1]:.6f} s", flush=True)
    drying_time_s = float(
        endpoint_times[str(args.fine_radial_intervals)]
        + (endpoint_times[str(args.fine_radial_intervals)] -
        endpoint_times[str(args.coarse_radial_intervals)]) / 3.0
    )
    timing_sensitivity: dict[str, float] = {}
    sensitivity_grid = max(160, args.coarse_radial_intervals // 2)
    for name, shift_s in (("half_interval_early", -900.0), ("baseline", 0.0), ("half_interval_late",
    900.0)):
        shifted_times = radius_history.time_s.copy()
        shifted_times[1:-1] += shift_s
        shifted_radius = RadiusHistory(shifted_times, radius_history.radius_m.copy())
        sensitivity_solution = solve_problem4(
            extended_chamber,
            shifted_radius,
            sensitivity_grid,
            probe_times,
            maximum_moisture_threshold=threshold,
            **solver_options,
        )
        timing_sensitivity[name] = float(sensitivity_solution.time_s[-1])
    if not (0.0 < drying_time_s < horizon_s):
        raise RuntimeError("Extrapolated drying endpoint is outside the supplied chamber horizon")
    strict_time_s = float(np.floor(drying_time_s / 60.0 + 1.0) * 60.0)
    if strict_time_s <= drying_time_s + 1.0e-8:
        strict_time_s += 60.0

    regular_times = np.arange(60.0, strict_time_s + 1.0e-9, 60.0)
    bracket = np.array([drying_time_s - 60.0, drying_time_s + 60.0])
    integration_times = np.unique(np.concatenate([regular_times, bracket]))
    fixed_radius_cm = np.round(np.arange(0.0, 2.0 + 0.0001, 0.1), 10)
    sampled_by_grid: list[np.ndarray] = []
    surface_by_grid: list[np.ndarray] = []
    balance_by_grid: dict[str, dict[str, float]] = {}
    monotonicity_by_grid: dict[str, float] = {}
    diagnostic_times = np.unique(np.concatenate([np.arange(0.0, 601.0), integration_times]))
    for grid in (args.coarse_radial_intervals, args.fine_radial_intervals):
        full = solve_problem4(
            extended_chamber,
            radius_history,
            grid,
            diagnostic_times,
            **solver_options,

```

```

    )
    balance_by_grid[str(grid)] = moisture_balance_diagnostics(
        full, extended_chamber, radius_history
    )
    monotonicity_by_grid[str(grid)] = validate_radially_nonincreasing_moisture(full)
    sampled_full = sample_moving_solution(full, radius_history, fixed_radius_cm)
    surface_full = sample_surface(full)
    retained = np.searchsorted(diagnostic_times, integration_times)
    sampled_by_grid.append(sampled_full[retained])
    surface_by_grid.append(surface_full[retained])
    del full
    print(f"full output grid={grid} complete", flush=True)

    extrapolated = richardson(sampled_by_grid[0], sampled_by_grid[1])
    extrapolated_surface = richardson(surface_by_grid[0], surface_by_grid[1])
    critical_fixed = interpolate_rows(extrapolated, integration_times, drying_time_s)
    critical_surface = float(interpolate_rows(extrapolated_surface[:, None], integration_times,
drying_time_s)[0])
    strict_index = int(np.where(np.isclose(integration_times, strict_time_s, atol=1.0e-9,
rtol=0.0))[0][0])
    strict_fixed = extrapolated[strict_index]
    strict_surface = float(extrapolated_surface[strict_index])
    regular_before = regular_times[regular_times < drying_time_s - 1.0e-8]
    output_times = np.concatenate([regular_before, [drying_time_s, strict_time_s]])
    output_fixed = np.vstack([
        extrapolated[np.searchsorted(integration_times, regular_before)],
        critical_fixed,
        strict_fixed,
    ])
    output_surface = np.concatenate([
        extrapolated_surface[np.searchsorted(integration_times, regular_before)],
        [critical_surface, strict_surface],
    ])

    endpoint_moisture = float(np.nanmax(critical_fixed))
    strict_maximum = float(np.nanmax(strict_fixed))
    if not np.isclose(endpoint_moisture, threshold, atol=1.0e-7, rtol=0.0):
        raise RuntimeError(f"Critical moisture does not equal threshold: {endpoint_moisture}")
    if not strict_maximum < threshold:
        raise RuntimeError(f"Strict regular row is not below threshold: {strict_maximum}")
    rounded_fixed = np.round(output_fixed, 4)
    workbook_matrix = [
        [None if not np.isfinite(value) else float(value) for value in row]
        for row in rounded_fixed
    ]
    payload = {
        "time_s": [int(round(t)) if np.isclose(t, round(t), rtol=0.0, atol=1.0e-8) else round(float(t),
6) for t in output_times],
        "radius_cm": fixed_radius_cm.tolist(),
        "moisture_concentration": workbook_matrix,
        "surface_moisture_concentration": np.round(output_surface, 4).tolist(),
        "drying_time_s": round(drying_time_s, 6),
        "drying_time_h": round(drying_time_s / 3600.0, 10),
        "strict_completion_time_s": int(round(strict_time_s)),
        "strict_completion_time_h": round(strict_time_s / 3600.0, 10),
        "threshold": threshold,
        "critical_maximum_moisture_unrounded": endpoint_moisture,
        "strict_completion_maximum_moisture_unrounded": strict_maximum,
        "radius_at_drying_time_cm": float(np.interp(drying_time_s, radius_history.time_s,
radius_history.radius_m) * 100.0),
        "grid_endpoint_s": endpoint_times | {"richardson": drying_time_s},
        "outside_radius_cells_are_blank": True,
        "moisture_balance": balance_by_grid,
        "radial_monotonicity": monotonicity_by_grid,
        "radius_timing_sensitivity_s": timing_sensitivity,
    }
    args.output.parent.mkdir(parents=True, exist_ok=True)
    args.output.write_text(
        json.dumps(payload, ensure_ascii=False, separators=(",", ":"), allow_nan=False),
        encoding="utf-8",

```

```

    )
    write_diagnostics(payload, args.diagnostics_output, args.chamber, args.radius)
    save_figures(output_times, fixed_radius_cm, output_fixed, output_surface, radius_history,
drying_time_s, args.figure_dir)
    print(f"drying endpoint: {drying_time_s / 3600.0:.6f} h; strict row: {strict_time_s:.0f} s",
flush=True)

if __name__ == "__main__":
    main()

```

verify_problem4.py

```

"""Reproduce the Q4 review experiments; no external review values are inputs."""

from __future__ import annotations

import argparse
from dataclasses import replace
import hashlib
import json
from pathlib import Path
import platform

import numpy as np
import scipy

from problem1 import Problem1Parameters, load_chamber_history_csv
from problem3 import extend_chamber_history_to_plateau
from problem4 import RadiusHistory, density_shrinkage_compatibility, load_radius_history_csv,
solve_problem4

def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--output", type=Path, default=Path("problem4_review_experiments.json"))
    parser.add_argument("--sensitivity-grid", type=int, default=320)
    args = parser.parse_args()
    chamber_path = Path("attachment1.csv")
    radius_path = Path("attachment2.csv")
    chamber = extend_chamber_history_to_plateau(load_chamber_history_csv(chamber_path), 144 * 3600.0)
    radius = load_radius_history_csv(radius_path)
    parameters = Problem1Parameters()
    results = {
        "schema_version": 1,
        "runtime": {"python": platform.python_version(), "numpy": np.__version__, "scipy":
scipy.__version__},
        "input_sha256": {str(p): hashlib.sha256(p.read_bytes()).hexdigest() for p in (chamber_path,
radius_path)},
        "threshold": 0.15,
        "default_solver": {"relative_tolerance": 2e-8, "max_step_s": 120.0},
        "sensitivity_radial_intervals": args.sensitivity_grid,
    }

    def save() -> None:
        args.output.parent.mkdir(parents=True, exist_ok=True)
        args.output.write_text(json.dumps(results, indent=2, ensure_ascii=False, allow_nan=False) +
"\n", encoding="utf-8")

    def endpoint(grid, radius_input=radius, params=parameters, rtol=2e-8, step=120.0):
        solution = solve_problem4(chamber, radius_input, grid, [0.0, 144 * 3600.0], parameters=params,
relative_tolerance=rtol, max_step_s=step,
maximum_moisture_threshold=0.15)
        return solution

    fixed = RadiusHistory(radius.time_s, np.full_like(radius.radius_m, parameters.radius_m))
    fixed_endpoints = {}
    results["fixed_radius_endpoint_s"] = fixed_endpoints

```

```

for grid in (160, 320, 640, 1280):
    fixed_endpoints[str(grid)] = float(endpoint(grid, fixed).time_s[-1])
    print(f"fixed radius N={grid}: {fixed_endpoints[str(grid)] / 3600:.9f} h", flush=True)
    save()
    fixed_endpoints["richardson_640_1280"] = fixed_endpoints["1280"] + (fixed_endpoints["1280"] -
fixed_endpoints["640"]) / 3
    fixed_endpoints["observed_order_320_640_1280"] = float(np.log2((fixed_endpoints["320"] -
fixed_endpoints["640"]) / (fixed_endpoints["640"] - fixed_endpoints["1280"])))

    time_checks = []
    results["time_integration"] = time_checks
    for rtol, step in ((2e-8, 120.0), (2e-9, 60.0), (5e-10, 30.0)):
        solution = endpoint(args.sensitivity_grid, rtol=rtol, step=step)
        time_checks.append({"relative_tolerance": rtol, "max_step_s": step, "endpoint_s":
float(solution.time_s[-1])})
        print(f"time convergence: {time_checks[-1]}", flush=True)
        save()
        baseline = time_checks[0]["endpoint_s"]
        results["time_integration_endpoint_spread_s"] = max(row["endpoint_s"] for row in time_checks) -
min(row["endpoint_s"] for row in time_checks)

        pchip = RadiusHistory(radius.time_s, radius.radius_m, interpolation="pchip")
        pchip_time = float(endpoint(args.sensitivity_grid, pchip).time_s[-1])
        results["radius_interpolation"] = {"linear_endpoint_s": baseline, "pchip_endpoint_s": pchip_time,
            "pchip_minus_linear_s": pchip_time - baseline,
"observation_times_unchanged": True}
        save()
        print(f"PCHIP minus linear: {pchip_time - baseline:.6f} s", flush=True)

        for name, field in (("mass_transfer", "mass_transfer_coefficient_m_s"), ("heat_transfer",
"heat_transfer_coefficient_w_m2_k")):
            rows = []
            results[name + "_sensitivity"] = rows
            for multiplier in (0.8, 1.0, 1.2):
                value = getattr(parameters, field) * multiplier
                time = baseline if multiplier == 1.0 else float(endpoint(args.sensitivity_grid,
params=replace(parameters, **{field: value})).time_s[-1])
                rows.append({"multiplier": multiplier, "coefficient": value, "endpoint_s": time,
"change_from_baseline_s": time - baseline})
            print(f"{name} x{multiplier}: {time / 3600:.9f} h", flush=True)
            save()

            density_checks = {}
            results["density_shrinkage_compatibility"] = density_checks
            for grid in (640, 1280):
                event_solution = endpoint(grid)
                times = np.append(np.arange(0.0, 49 * 3600.0, 6 * 3600.0), event_solution.time_s[-1])
                full = solve_problem4(chamber, radius, grid, times, relative_tolerance=2e-8, max_step_s=120.0)
                density_checks[str(grid)] = density_shrinkage_compatibility(full, radius)
                density_checks[str(grid)]["center_moisture_at_48h"] = float(full.moisture_concentration[-2, 0])
                print(f"density compatibility N={grid}: final mass ratio
{density_checks[str(grid)]['final_mass_ratio']:.9f}", flush=True)
                save()
            save()

if __name__ == "__main__":
    main()

```

verify_problem4_independent.py

"""Reproduce a separately implemented cell-centred discretization of Problem 4."""

from __future__ import annotations

```

import argparse
import json
from pathlib import Path

```

```

import platform
import time

import numpy as np
import scipy

from problem1 import load_chamber_history_csv
from problem3 import extend_chamber_history_to_plateau
from problem4 import load_radius_history_csv
from problem4_independent import (
    sample_independent_moisture,
    solve_independent_problem4,
)

def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--cells", nargs="+", type=int, default=[100, 200, 400, 800, 1600, 3200])
    parser.add_argument("--output", type=Path, default=Path("problem4_independent_validation.json"))
    args = parser.parse_args()
    raw_chamber = load_chamber_history_csv("attachment1.csv")
    chamber = extend_chamber_history_to_plateau(raw_chamber, 72 * 3600.0)
    radius = load_radius_history_csv("attachment2.csv")
    table_hours = np.arange(6.0, 48.1, 6.0)
    requested_times = np.concatenate([[0.0], table_hours * 3600.0, [72 * 3600.0]])
    reference_table = np.asarray([
        [1.7196, 1.5377, 1.0226, 0.4207],
        [0.7377, 0.6546, 0.4083, 0.1669],
        [0.4088, 0.3687, 0.2397, 0.0892],
        [0.2851, 0.2611, 0.1790, 0.0673],
        [0.2263, 0.2094, 0.1493, 0.0595],
        [0.1928, 0.1797, 0.1317, 0.0560],
        [0.1712, 0.1604, 0.1200, 0.0541],
        [0.1561, 0.1468, 0.1116, 0.0530],
    ])
    grids = {}
    for cells in args.cells:
        started = time.perf_counter()
        solution = solve_independent_problem4(chamber, radius, cells, requested_times)
        sampled = sample_independent_moisture(solution, radius, [0.0, 0.5, 1.0])
        table = np.column_stack([sampled[1:9], solution.surface_moisture[1:9]])
        full_profiles = np.column_stack([
            solution.centre_moisture, solution.moisture, solution.surface_moisture,
        ])
        grids[str(cells)] = {
            "cells": cells,
            "drying_time_s": solution.drying_time_s,
            "drying_time_h": solution.drying_time_s / 3600.0,
            "endpoint_cell_maximum": solution.endpoint_cell_maximum,
            "endpoint_reconstructed_maximum": solution.endpoint_reconstructed_maximum,
            "endpoint_centre_moisture": float(solution.centre_moisture[-1]),
            "endpoint_surface_moisture": float(solution.surface_moisture[-1]),
            "table_unrounded": table.tolist(),
            "table_rounded_4dp": np.round(table, 4).tolist(),
            "maximum_outward_increase_at_stored_times": float(np.max(np.diff(full_profiles, axis=1))),
            "elapsed_s": time.perf_counter() - started,
        }
    print(f"cells={cells}: endpoint {solution.drying_time_s:.9f} s = "
          f"{solution.drying_time_s / 3600:.10f} h", flush=True)

    output = {
        "purpose": "Numerical cross-check of the same assumed continuum model; not independent experimental validation.",
        "method": {
            "geometry": "N complete annular cells; point unknowns at  $\xi=(j+1/2)/N$ ",
            "internal_faces": "Arithmetic mean transport coefficient with centred gradient",
            "outer_boundary": "Last-centre half-cell resistance  $R/(2*N*a_{last})$  plus convection resistance  $1/h$ ",
            "centre_reconstruction": "(9*C[0]-C[1])/8 from an even quadratic",
            "event": "Maximum of reconstructed centre, all cell-centre values and reconstructed surface

```

```

equals 0.15",
    "time_integrator": "SciPy BDF for T and C directly; rtol=2e-9, atol(T)=1e-9, atol(C)=1e-11,
max_step=60s",
    "material_laws": "Independent transcription of Appendix 4 with Kelvin in diffusivity",
    "shared_inputs": "Same Attachment 1 and 2 CSV inputs, linear radius and chamber
interpolation, final-hour chamber means with 60s transition",
    "limitation": "Arithmetic-face and frozen half-cell coefficient errors, and interior linear
sampling, require convergence checks. The reconstruction is not a rigorous bound on continuous subcell
maxima.",
    },
    "runtime": {"python": platform.python_version(), "numpy": np.__version__, "scipy":
scipy.__version__},
    "chamber_plateau": {"temperature_c": float(chamber.temperature_c[-1]), "moisture":
float(chamber.moisture_concentration[-1])},
    "table_time_h": table_hours.tolist(),
    "table_columns": ["0 cm", "0.5 cm", "1.0 cm", "actual surface"],
    "baseline_report": {"drying_time_h": 51.091955, "table_6_rounded_4dp":
reference_table.tolist()},
    "unverified_docx_reference_time_h": 51.091999,
    "grids": grids,
    }
    extrapolations = {}
    for coarse in args.cells:
        fine = 2 * coarse
        if str(fine) not in grids:
            continue
        coarse_value, fine_value = grids[str(coarse)], grids[str(fine)]
        extrap_time = fine_value["drying_time_s"] + (
            fine_value["drying_time_s"] - coarse_value["drying_time_s"]
        ) / 3.0
        extrap_table = np.asarray(fine_value["table_unrounded"]) + (
            np.asarray(fine_value["table_unrounded"]) - np.asarray(coarse_value["table_unrounded"])
        ) / 3.0
        rounded = np.round(extrap_table, 4)
        mismatches = np.argwhere(rounded != reference_table)
        extrapolations[f"{coarse}/{fine}"] = {
            "assumed_order": 2,
            "drying_time_s": extrap_time,
            "drying_time_h": extrap_time / 3600.0,
            "delta_from_baseline_s": extrap_time - 51.091955 * 3600.0,
            "delta_from_unverified_docx_reference_s": extrap_time - 51.091999 * 3600.0,
            "table_unrounded": extrap_table.tolist(),
            "table_rounded_4dp": rounded.tolist(),
            "maximum_absolute_difference_from_rounded_table_6": float(np.max(np.abs(extrap_table -
reference_table))),
            "rounded_table_mismatch_count": int(len(mismatches)),
            "rounded_table_mismatches": [
                {"time_h": float(table_hours[row]), "column": output["table_columns"][col],
                "baseline": float(reference_table[row, col]), "independent": float(rounded[row, col])}
                for row, col in mismatches
            ],
        }
    output["second_order_richardson"] = extrapolations
    orders = {}
    for coarse in args.cells:
        if str(2 * coarse) in grids and str(4 * coarse) in grids:
            a, b, c = [grids[str(k)]["drying_time_s"] for k in (coarse, 2 * coarse, 4 * coarse)]
            orders[f"{coarse}/{2 * coarse}/{4 * coarse}"] = float(np.log2(abs((a - b) / (b - c))))
    output["observed_endpoint_orders"] = orders
    output["interpretation"] = (
        "The endpoint differences decrease with refinement, but observed orders remain below two. "
        "The second-order Richardson values are diagnostic extrapolations, not certified error bounds.
"
        "Agreement with a production-solver extrapolation does not validate the shared physical
assumptions. "
        "Four-decimal table agreement is assessed against the rounded published table; its raw
differences "
        "therefore include reference rounding and are not discretization error estimates."
    )
    args.output.parent.mkdir(parents=True, exist_ok=True)

```

```
args.output.write_text(json.dumps(output, indent=2, ensure_ascii=False, allow_nan=False) + "\n",
encoding="utf-8")
print(json.dumps({"extrapolated_endpoints_h": {key: row["drying_time_h"] for key, row in
extrapolations.items()},
"observed_orders": orders}, ensure_ascii=False), flush=True)

if __name__ == "__main__":
    main()
```